

EEE 475/575 – Medical Image Reconstruction

Homework #2

Özgür Ekin Sönmez 22201966

31 March 2026

Contents

GenAI Usage Disclosure	2
1 Part I: Gridding Reconstruction (Spiral Data)	2
1.1 K-Space Trajectory (1.1)	2
1.2 Density Compensation via Voronoi (1.2)	2
1.3 Correcting the DCF (1.3)	3
1.4 Direct Summation Reference (1.4)	4
1.5 No Density Compensation (1.5)	5
1.6 Double FOV (1.6)	6
1.7 Half Pixel Size (1.7)	8
1.8 Double Pixel Size (1.8)	9
1.9 1X Gridding (1.9)	10
1.10 2X Gridding (1.10)	12
1.11 Scattered Interpolation 1X (1.11)	13
1.12 Scattered Interpolation 2X (1.12)	14
2 Part II: Gridding vs Backprojection	15
2.1 Phantom and Sinogram (2.1)	15
2.2 Filtered Backprojection (2.2)	16
2.3 Naive Backprojection (2.3)	18
2.4 K-Space via Projection-Slice Theorem (2.4)	19
2.5 Geometric DCF (2.5)	19
2.6 Direct Summation - Projection Data (2.6)	20
2.7 2X Gridding - Projection Data (2.7)	21
3 Part III: Gridding for Radial Data	23
3.1 K-Space Trajectory (3.1)	23
3.2 Geometric DCF (3.2)	23
3.3 Direct Summation (3.3)	24
3.4 2X Gridding (3.4)	25
4 Results Summary	27

GenAI Usage Disclosure

I used Google Gemini (Antigravity) to help me structure the MATLAB code and create some parts like setup to add plot saving and adjust figure scalings. I also used Codex (GPT 5.4) to help me to format this Latex report. I checked all results myself and I can explain every part.

1 Part I: Gridding Reconstruction (Spiral Data)

1.1 K-Space Trajectory (1.1)

The spiral k-space data consists of 6 interleaves with 2048 samples each. The trajectory spirals outward from the k-space center, giving denser sampling near DC and sparser sampling at higher spatial frequencies.

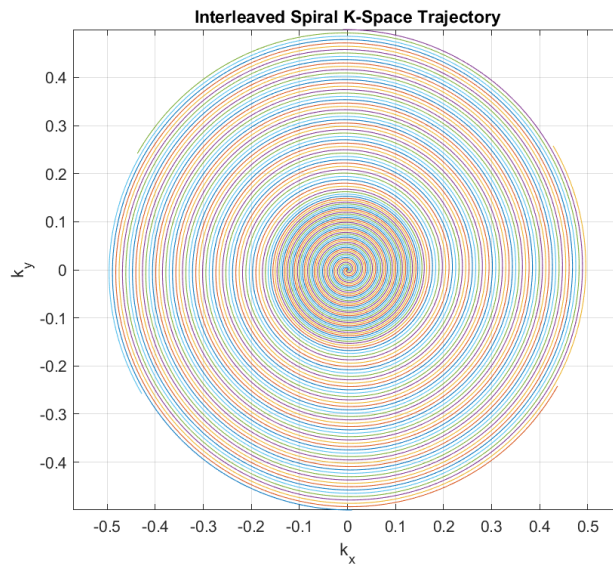
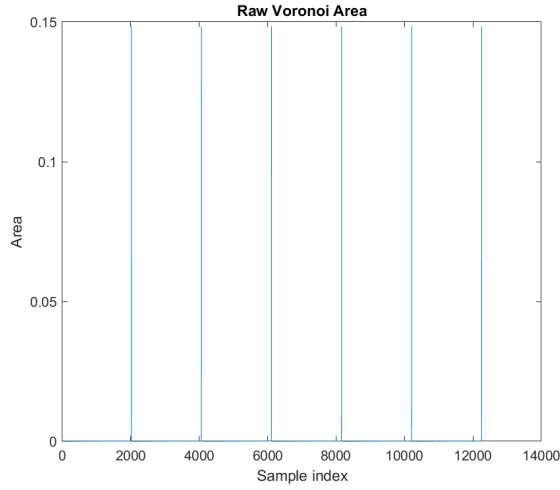


Figure 1: Spiral k-space trajectory.

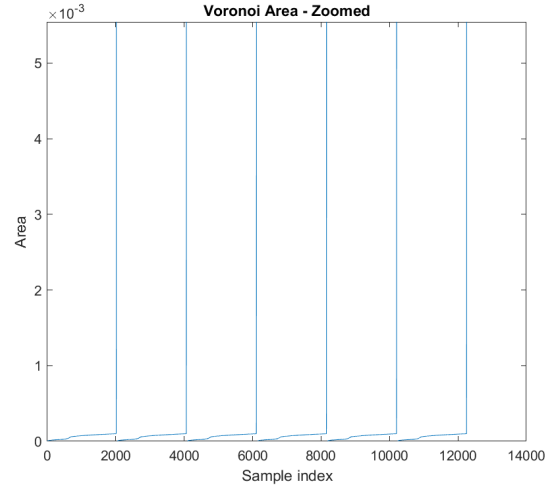
```
1 load('spiral.mat');
2 figure; plot(ktraj); xlabel('k_x'); ylabel('k_y');
3 title('Interleaved Spiral K-Space Trajectory'); axis equal; grid on;
4 saveas(gcf, fullfile(plotDir, '1_1_spiral_trajectory.png'));
```

1.2 Density Compensation via Voronoi (1.2)

The Voronoi tessellation assigns an area to each sample point. 186 samples at the boundary have NaN (unbounded cells). Some edge samples also have unreasonably large areas because their Voronoi cells extend far beyond the trajectory.



(a) Raw.



(b) Zoomed.

Figure 2: Voronoi areas.

```

1 area_raw = voronoidens(ktraj);
2 nan_count = sum(isnan(area_raw(:)));
3 fprintf('NaN count: %d\n', nan_count);
4 figure; plot(area_raw(:)); xlabel('Sample index'); ylabel('Area');
5 title('Raw Voronoi Area'); saveas(gcf, fullfile(plotDir, '1_2_raw_area.png'));
6 valid = area_raw(~isnan(area_raw(:)) & area_raw(:)<1e-2);
7 figure; plot(area_raw(:)); ylim([0 max(valid)*1.5]);
8 xlabel('Sample index'); ylabel('Area'); title('Voronoi Area - Zoomed');
9 saveas(gcf, fullfile(plotDir, '1_2_raw_area_zoomed.png'));

```

1.3 Correcting the DCF (1.3)

I replaced NaN and outlier values with a threshold of $3 \times \text{median of finite values}$ (≈ 0.000222). This prevents boundary artifacts while preserving the correct weighting for interior samples.

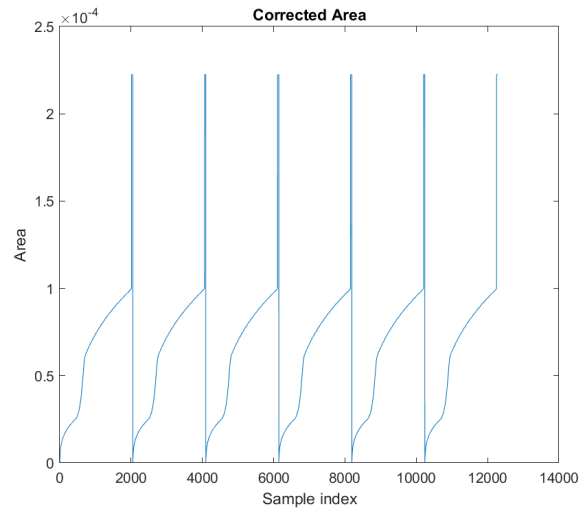


Figure 3: Corrected DCF.

```

1 area_corr = area_raw;
2 fin = area_raw(isfinite(area_raw(:)));
3 thr = 3*median(fin);
4 area_corr(isnan(area_corr)) = thr;

```

```

5 area_corr(area_corr > thr) = thr;
6 fprintf('Threshold: %.6f, remaining NaN: %d\n', thr, sum(isnan(area_corr(:))));
7 figure; plot(area_corr(:)); xlabel('Sample index'); ylabel('Area');
8 title('Corrected Area'); saveas(gcf, fullfile(plotDir, '1_3_corrected_area.png'));

```

1.4 Direct Summation Reference (1.4)

Implements the 2D NUFFT via direct summation: $m[n_x, n_y] = \sum_{\ell} d_{\ell} M(k_{\ell}) e^{j2\pi(k_{x,\ell}\Delta x(n_x - \tau) + k_{y,\ell}\Delta x(n_y - \tau))}$ with $N = 128$, $\Delta x = 1$, $\tau = 64$. I loop row-by-row to keep memory low. This is the reference image for Part I. **CPU: 0.98 s.**

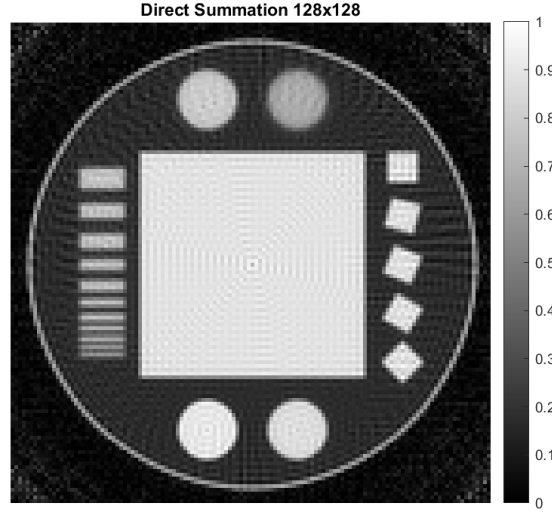
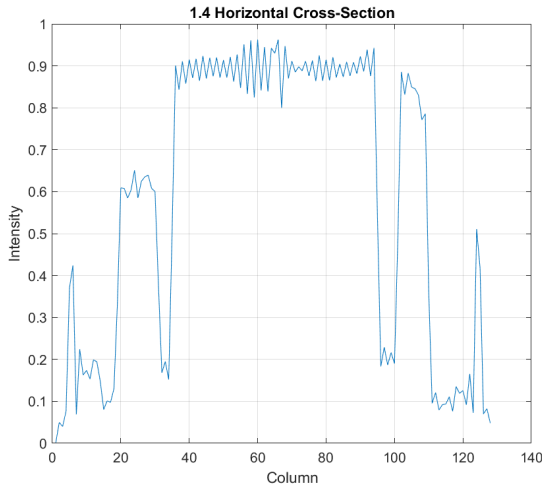
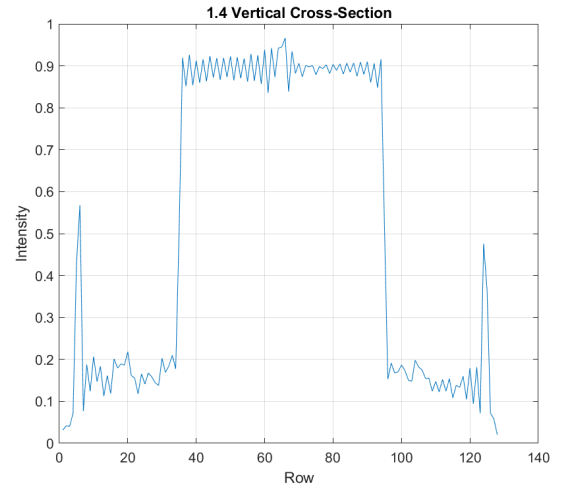


Figure 4: Reference image.



(a) Horizontal.



(b) Vertical.

Figure 5: Cross-sections.

```

1 N=128; dx=1; tau=N/2;
2 kx_vec=real(ktraj(:)); ky_vec=imag(ktraj(:));
3 M_vec=kdata(:); d_vec=area_corr(:);
4 dM = d_vec.*M_vec;
5 n_vals = (0:N-1)-tau;

```

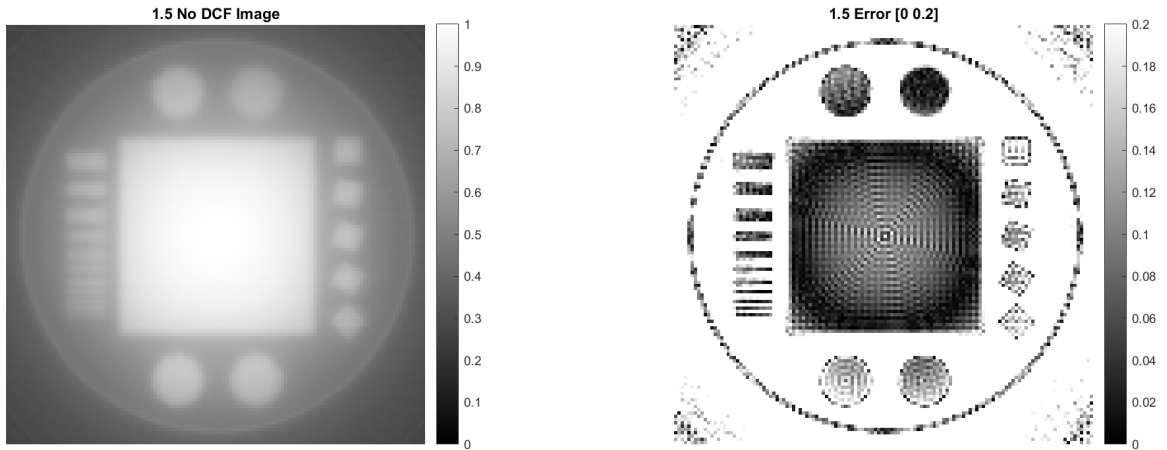
```

6 phase_y = exp(1j*2*pi*dx * ky_vec * n_vals);
7 ima_direct = zeros(N,N);
8 t0 = cputime;
9 for ix = 1:N
10     phase_x = exp(1j*2*pi*dx * kx_vec * n_vals(ix));
11     ima_direct(ix,:) = (dM.*phase_x).' * phase_y;
12 end
13 t_direct = cputime-t0; clear phase_y phase_x;
14 ima_direct = normImg(ima_direct. ');
15 fprintf('1.4 CPU time: %.2f s\n', t_direct);
16 figure; imshow(ima_direct,[0 1]); colormap gray; colorbar;
17 title('Direct Summation 128x128'); saveas(gcf, fullfile(plotDir, '1_4_image.png'));
18 figure; plot(abs(ima_direct(end/2,:))); xlabel('Column'); ylabel('Intensity');
19 title('1.4 Horizontal Cross-Section'); grid on; saveas(gcf, fullfile(plotDir, '1_4_horiz.png'));
20 figure; plot(abs(ima_direct(:,end/2))); xlabel('Row'); ylabel('Intensity');
21 title('1.4 Vertical Cross-Section'); grid on; saveas(gcf, fullfile(plotDir, '1_4_vert.png'));
22 save(fullfile(matDir, 'ima_direct.mat'), 'ima_direct');

```

1.5 No Density Compensation (1.5)

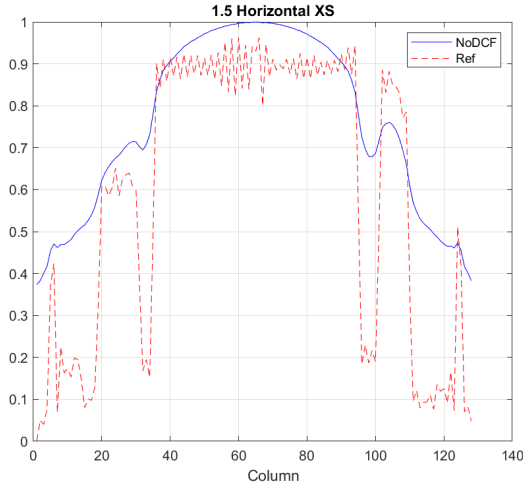
Without DCF, low frequencies are overweighted because the spiral samples more densely near the k-space center. The image appears blurred and washed out. **PSNR=10.86 dB**, **SSIM=0.2729**, **CPU=0.95 s**. This confirms that density compensation is essential for non-Cartesian reconstruction.



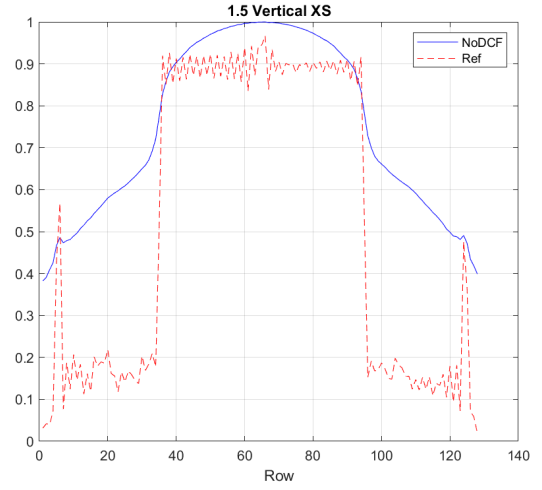
(a) Image.

(b) Error.

Figure 6: 1.5 No DCF.



(a) Horiz XS.



(b) Vert XS.

Figure 7: 1.5 Cross-sections.

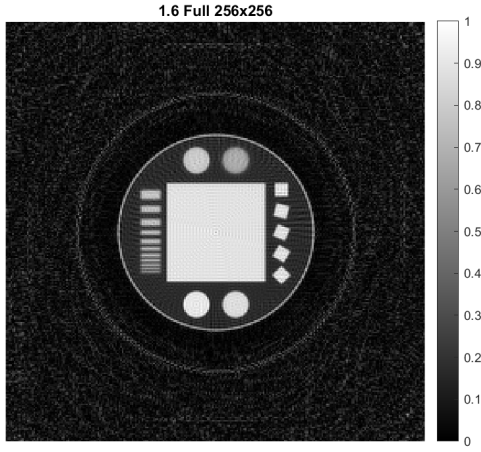
```

1  n_vals = (0:N-1)-tau;
2  phase_y = exp(1j*2*pi*dx * ky_vec * n_vals);
3  ima_nodcf = zeros(N,N);
4  t0 = cputime;
5  for ix = 1:N
6      phase_x = exp(1j*2*pi*dx * kx_vec * n_vals(ix));
7      ima_nodcf(ix,:) = (M_vec.*phase_x).' * phase_y;
8  end
9  t_nodcf = cputime-t0; clear phase_y phase_x;
10 ima_nodcf = normImg(ima_nodcf. ');
11 err_nodcf = abs(ima_nodcf - ima_direct);
12 psnr_nodcf = psnr(ima_nodcf, ima_direct);
13 ssim_nodcf = ssim(ima_nodcf, ima_direct);
14 fprintf('1.5 No DCF: PSNR=%.2f dB, SSIM=%.4f, CPU=%.2f s\n', psnr_nodcf, ssim_nodcf, t_nodcf);
15 figure; imshow(ima_nodcf,[0 1]); colormap gray; colorbar;
16 title('1.5 No DCF Image'); saveas(gcf, fullfile(plotDir, '1_5_image.png'));
17 figure; imshow(err_nodcf,[0 0.2]); colormap gray; colorbar;
18 title('1.5 Error [0 0.2]'); saveas(gcf, fullfile(plotDir, '1_5_error.png'));
19 figure; plot(abs(ima_nodcf(end/2,:)), 'b'); hold on;
20 plot(abs(ima_direct(end/2,:)), 'r--'); legend('NoDCF', 'Ref');
21 xlabel('Column'); title('1.5 Horizontal XS'); grid on; saveas(gcf, fullfile(plotDir, '1_5_horiz.
    png'));
22 figure; plot(abs(ima_nodcf(:,end/2)), 'b'); hold on;
23 plot(abs(ima_direct(:,end/2)), 'r--'); legend('NoDCF', 'Ref');
24 xlabel('Row'); title('1.5 Vertical XS'); grid on; saveas(gcf, fullfile(plotDir, '1_5_vert.png'))
    ;

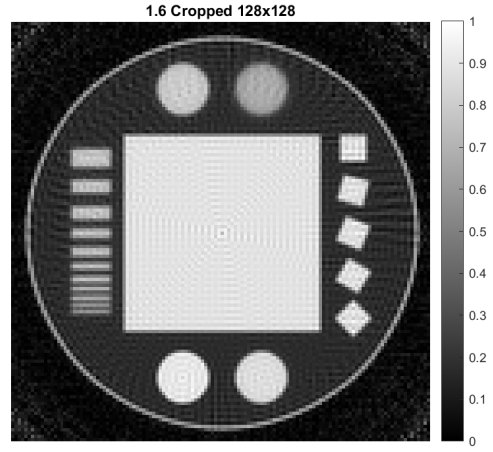
```

1.6 Double FOV (1.6)

With $N = 256$, $\Delta x = 1$, FOV doubles. The object appears in the center with empty space around it. The cropped 128×128 center is identical to the reference (PSNR= ∞ , SSIM=1.0000). No new information is gained but computation took 2.36 s vs 0.98 s. **CPU=2.36 s.**

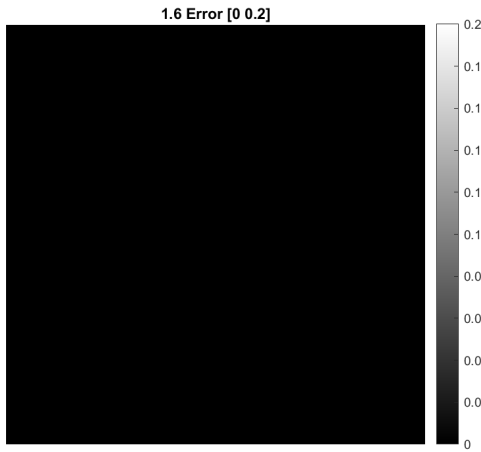


(a) Full 256x256.

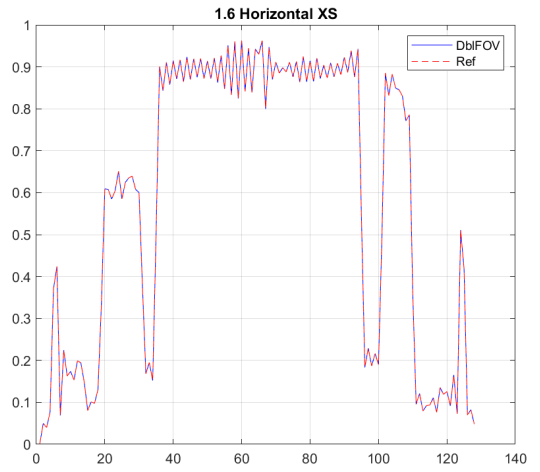


(b) Cropped 128x128.

Figure 8: 1.6 Double FOV.



(a) Error.



(b) Horizontal XS.

Figure 9: 1.6 Error and horizontal cross-section.

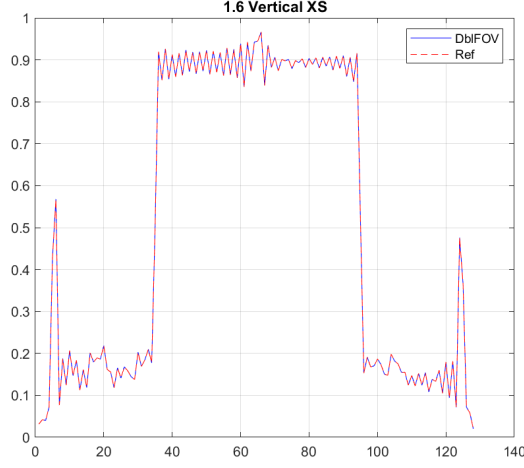


Figure 10: 1.6 Vertical cross-section.

```

1 N2=256; tau2=N2/2; n2=(0:N2-1)-tau2;
2 phase_y2 = exp(1j*2*pi*dx * ky_vec * n2);
3 ima_dfov = zeros(N2,N2);
4 t0 = cputime;
5 for ix = 1:N2
6     px = exp(1j*2*pi*dx * kx_vec * n2(ix));
7     ima_dfov(ix,:) = (dM.*px).' * phase_y2;
8 end
9 t_dfov = cputime-t0; clear phase_y2 px;
10 ima_dfov = normImg(ima_dfov.');
```

```

11 c=N2/2; h=N/2;
12 ima_dfov_crop = normImg(ima_dfov(c-h+1:c+h, c-h+1:c+h));
13 err_dfov = abs(ima_dfov_crop - ima_direct);
14 psnr_dfov=psnr(ima_dfov_crop,ima_direct); ssim_dfov=ssim(ima_dfov_crop,ima_direct);
15 fprintf('1.6: PSNR=%.2f, SSIM=%.4f, CPU=%.2f\n',psnr_dfov,ssim_dfov,t_dfov);
16 figure; imshow(ima_dfov,[0 1]); colormap gray; colorbar;
17 title('1.6 Full 256x256'); saveas(gcf, fullfile(plotDir,'1_6_full.png'));
18 figure; imshow(ima_dfov_crop,[0 1]); colormap gray; colorbar;
19 title('1.6 Cropped 128x128'); saveas(gcf, fullfile(plotDir,'1_6_crop.png'));
20 figure; imshow(err_dfov,[0 0.2]); colormap gray; colorbar;
21 title('1.6 Error'); saveas(gcf, fullfile(plotDir,'1_6_error.png'));
22 figure; plot(abs(ima_dfov_crop(end/2,:)),'b'); hold on;
23 plot(abs(ima_direct(end/2,:)),'r--'); legend('DblFOV','Ref');
24 title('1.6 Horiz XS'); grid on; saveas(gcf, fullfile(plotDir,'1_6_horiz.png'));
25 figure; plot(abs(ima_dfov_crop(:,end/2)),'b'); hold on;
26 plot(abs(ima_direct(:,end/2)),'r--'); legend('DblFOV','Ref');
27 title('1.6 Vert XS'); grid on; saveas(gcf, fullfile(plotDir,'1_6_vert.png'));
```

1.7 Half Pixel Size (1.7)

With $\Delta x = 0.5$, $N = 256$, same FOV but finer pixels. Reference is zero-padded in k-space to 256×256 for fair comparison. The result is very close to the reference. **PSNR=33.67 dB**, **SSIM=0.9150**, **CPU=3.62 s**. Higher resolution image without new frequency content.

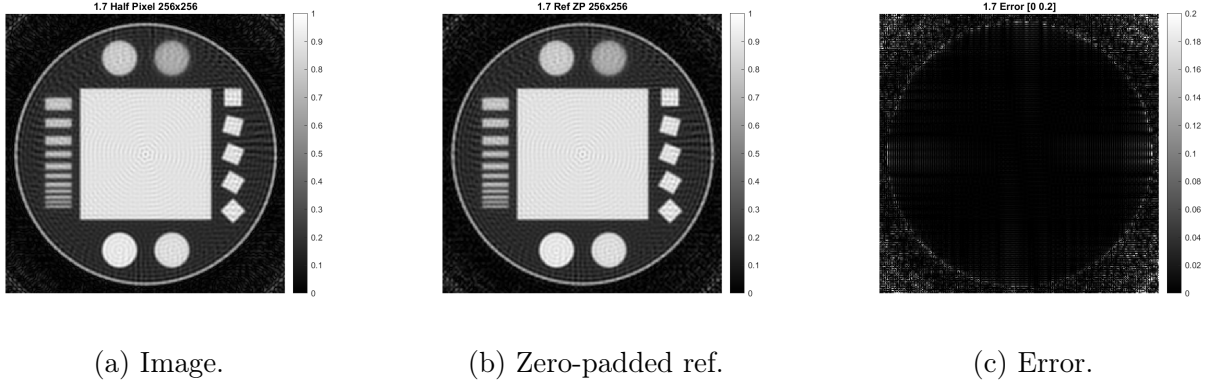


Figure 11: 1.7 Half pixel.

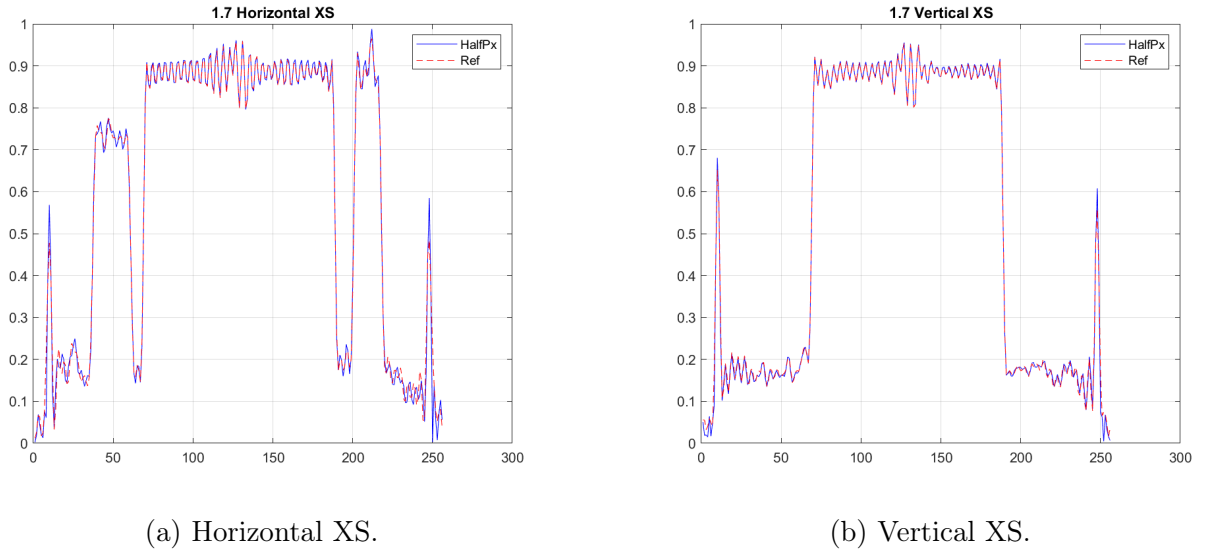


Figure 12: 1.7 Cross-sections.

```

1 N3=256; dx3=0.5; tau3=N3/2; n3=(0:N3-1)-tau3;
2 phase_y3 = exp(1j*2*pi*dx3 * ky_vec * n3);
3 ima_hpx = zeros(N3,N3);
4 t0=cputime;
5 for ix=1:N3
6     px=exp(1j*2*pi*dx3*kx_vec*n3(ix));
7     ima_hpx(ix,:)=(dM.*px).'*phase_y3;
8 end
9 t_hpx=cputime-t0; clear phase_y3 px;
10 ima_hpx = normImg(ima_hpx. ');
11 kref=fftshift(fft2(ifftshift(ima_direct)));
12 pad=(N3-N)/2; kref_pad=zeros(N3,N3);
13 kref_pad(pad+1:pad+N, pad+1:pad+N)=kref;
14 ima_ref256=normImg(fftshift(ifft2(ifftshift(kref_pad))));
15 err_hpx=abs(ima_hpx-ima_ref256);
16 psnr_hpx=psnr(ima_hpx,ima_ref256); ssim_hpx=ssim(ima_hpx,ima_ref256);
17 figure; imshow(ima_hpx,[0 1]); colormap gray; colorbar;
18 title('1.7 Half Pixel'); saveas(gcf, fullfile(plotDir,'1_7_image.png'));
19 figure; imshow(err_hpx,[0 0.2]); colormap gray; colorbar;
20 title('1.7 Error'); saveas(gcf, fullfile(plotDir,'1_7_error.png'));

```

1.8 Double Pixel Size (1.8)

With $\Delta x = 2$, $N = 64$, same FOV but coarser pixels. Result is zero-padded from 64×64 to 128×128 for comparison. Fine details are visibly lost. **PSNR=18.95 dB**, **SSIM=0.6160**, **CPU=0.36 s**. Faster but too much quality loss.

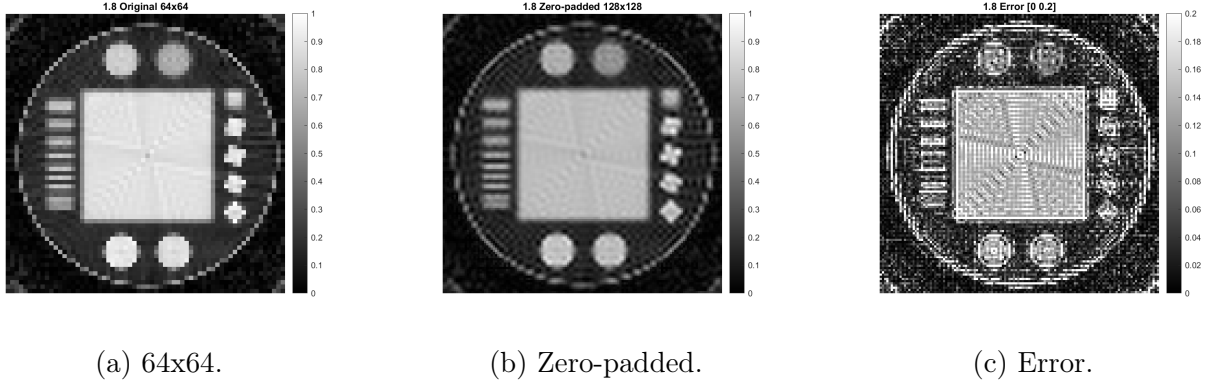


Figure 13: 1.8 Double pixel.

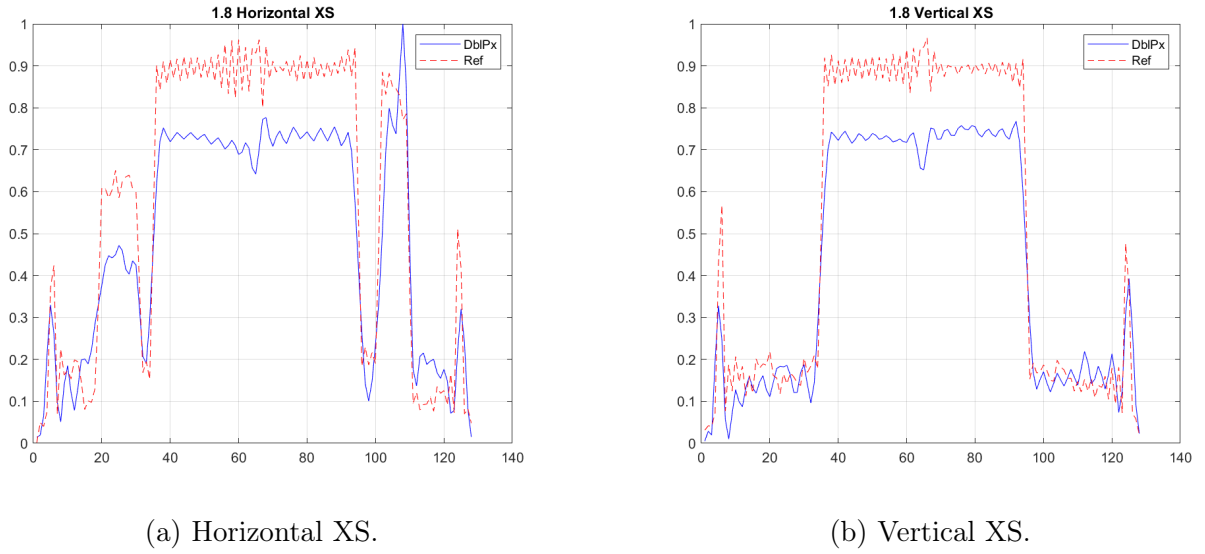


Figure 14: 1.8 Cross-sections.

```

1  N4=64; dx4=2; tau4=N4/2; n4=(0:N4-1)-tau4;
2  phase_y4=exp(1j*2*pi*dx4*ky_vec*n4);
3  ima_dpx=zeros(N4,N4);
4  t0=cputime;
5  for ix=1:N4
6      px=exp(1j*2*pi*dx4*kx_vec*n4(ix));
7      ima_dpx(ix,:)=(dM.*px).'*phase_y4;
8  end
9  t_dpx=cputime-t0; clear phase_y4 px;
10 ima_dpx=normImg(ima_dpx. ');
11 kdpd=fftshift(fft2(fftshift(ima_dpx)));
12 pad2=(N-N4)/2; kdpd_pad=zeros(N,N);
13 kdpd_pad(pad2+1:pad2+N4,pad2+1:pad2+N4)=kdpd;
14 ima_dpx_zp=normImg(fftshift(fft2(fftshift(kdpd_pad))));
15 err_dpx=abs(ima_dpx_zp-ima_direct);
16 psnr_dpx=psnr(ima_dpx_zp,ima_direct); ssim_dpx=ssim(ima_dpx_zp,ima_direct);
17 figure; imshow(ima_dpx,[0 1]); colormap gray; colorbar;
18 title('1.8_Original_64x64'); saveas(gcf, fullfile(plotDir,'1_8_image.png'));
19 figure; imshow(err_dpx,[0 0.2]); colormap gray; colorbar;
20 title('1.8_Error'); saveas(gcf, fullfile(plotDir,'1_8_error.png'));

```

1.9 1X Gridding (1.9)

Using `gridkb` with `osf=1`, `wg=2`. I transpose the gridding output to match the image-axis convention used by the direct-summation reference. The result is much closer to the reference than

before, but some residual artifacts remain because the grid is not oversampled. **PSNR=23.91 dB, SSIM=0.7537, CPU=0.05 s.**

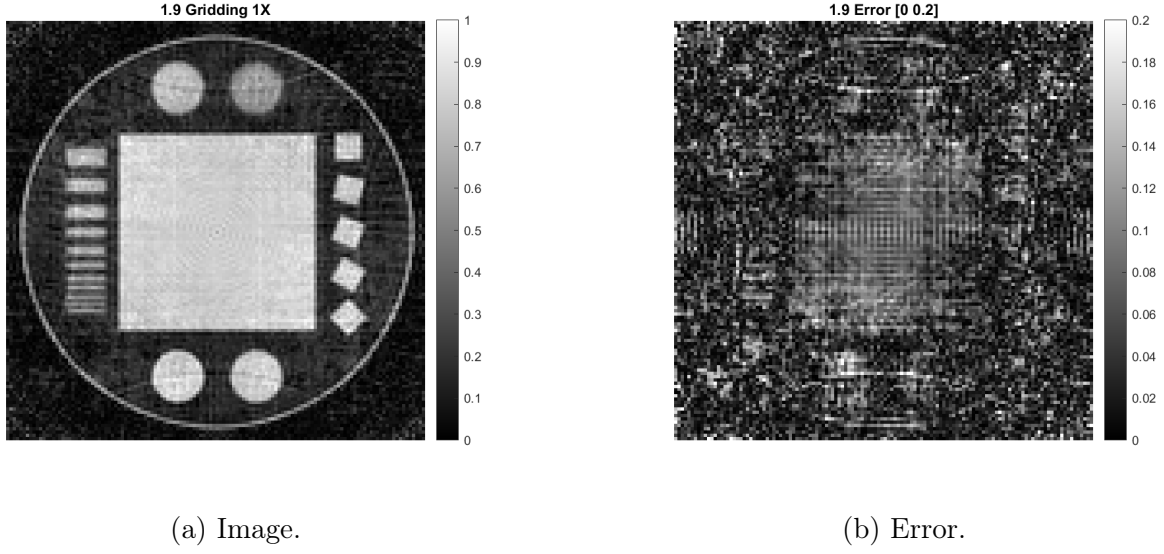


Figure 15: 1.9 Gridding 1X.

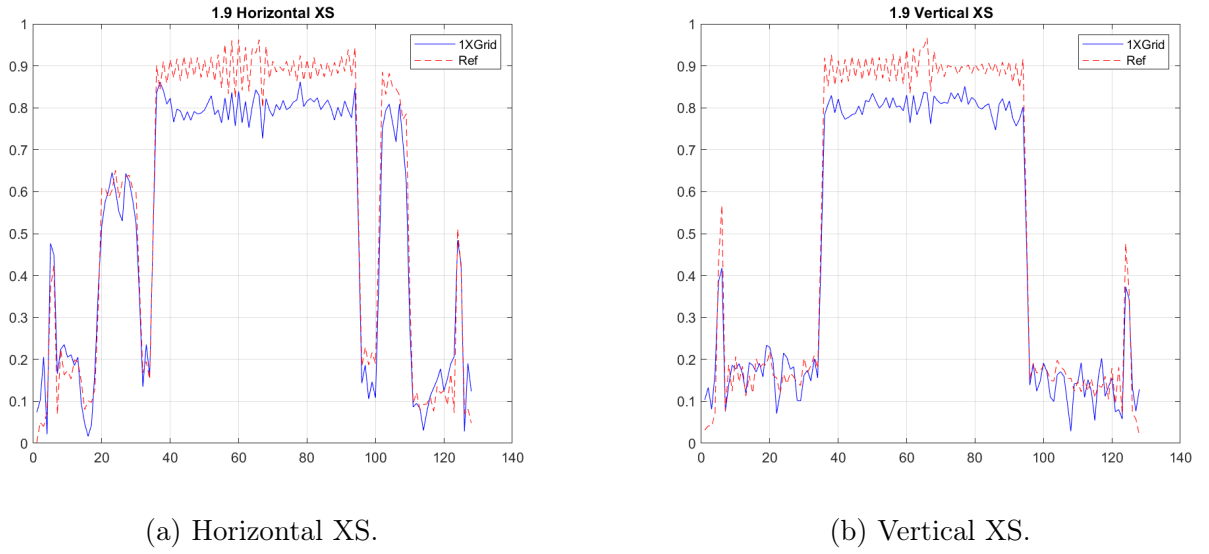


Figure 16: 1.9 Cross-sections.

```

1  t0=cputime;
2  ima_g1=gridkb(kdata,ktraj,area_corr,128,1,2,'image');
3  t_g1=cputime-t0;
4  ima_g1=normImg(ima_g1.'');
5  err_g1=abs(ima_g1-ima_direct);
6  psnr_g1=psnr(ima_g1,ima_direct); ssim_g1=ssim(ima_g1,ima_direct);
7  figure; imshow(ima_g1,[0 1]); colormap gray; colorbar;
8  title('1.9 Gridding 1X'); saveas(gcf, fullfile(plotDir,'1_9_image.png'));
9  figure; imshow(err_g1,[0 0.2]); colormap gray; colorbar;
10 title('1.9 Error'); saveas(gcf, fullfile(plotDir,'1_9_error.png'));
11 figure; plot(abs(ima_g1(end/2,:)), 'b'); hold on;
12 plot(abs(ima_direct(end/2,:)), 'r--'); legend('1XGrid', 'Ref');
13 title('1.9 Horiz XS'); grid on; saveas(gcf, fullfile(plotDir,'1_9_horiz.png'));
14 figure; plot(abs(ima_g1(:,end/2)), 'b'); hold on;
15 plot(abs(ima_direct(:,end/2)), 'r--'); legend('1XGrid', 'Ref');
16 title('1.9 Vert XS'); grid on; saveas(gcf, fullfile(plotDir,'1_9_vert.png'));

```

1.10 2X Gridding (1.10)

With $\text{osf}=2$, $\text{wg}=4$. Grid is 256×256 ; I transpose the gridding output to match the reference orientation, then crop the central 128×128 . The wider kernel and oversampling reduce aliasing strongly, and the result becomes nearly identical to the reference. **PSNR=44.35 dB**, **SSIM=0.9906**, **CPU=0.09 s**. This is a major improvement over 1X gridding.

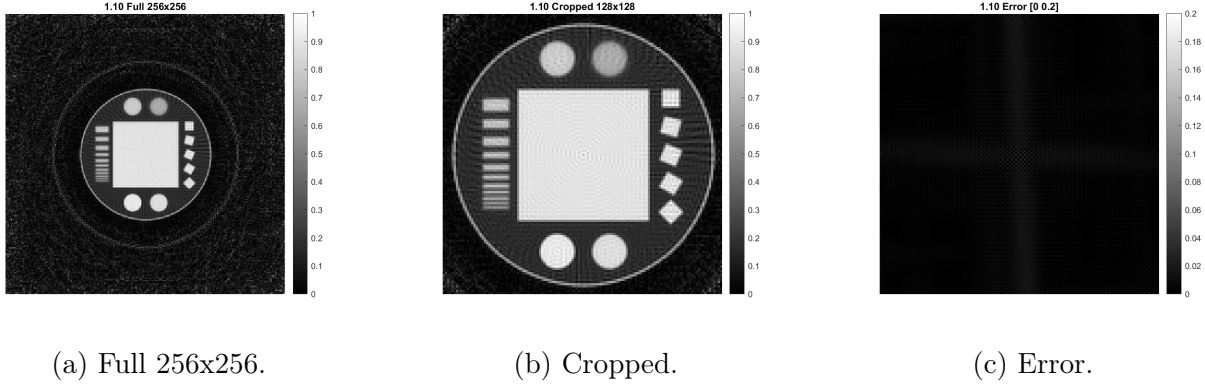


Figure 17: 1.10 Gridding 2X.

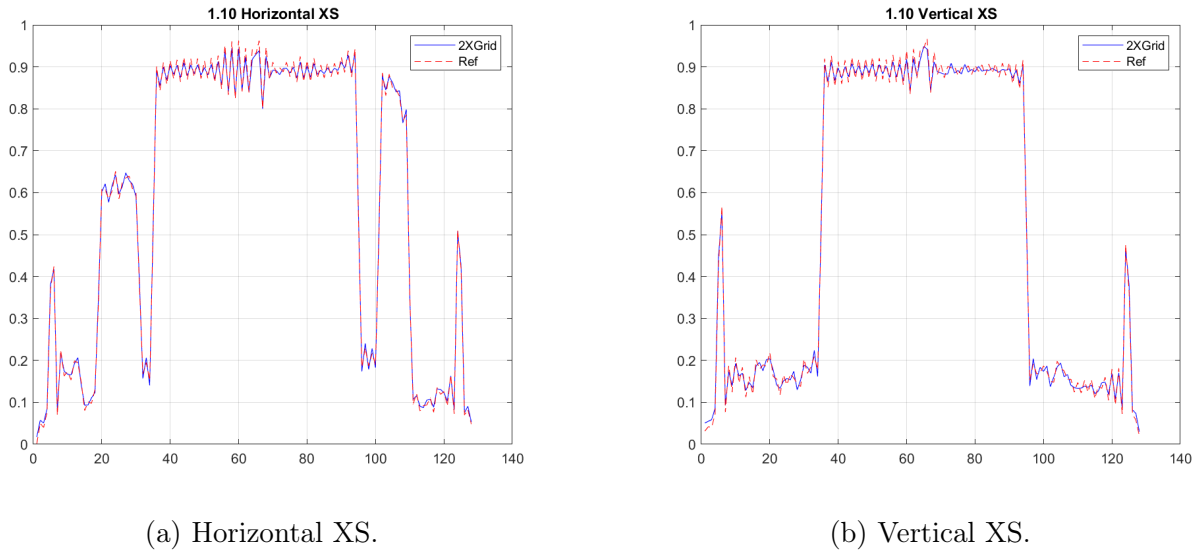


Figure 18: 1.10 Cross-sections.

```

1 t0=cputime;
2 ima_g2_full=gridkb(kdata,ktraj,area_corr,128,2,4,'image');
3 t_g2=cputime-t0;
4 ima_g2_full=normImg(ima_g2_full. ');
5 c2=128; h2=64;
6 ima_g2=normImg(ima_g2_full(c2-h2+1:c2+h2,c2-h2+1:c2+h2));
7 err_g2=abs(ima_g2-ima_direct);
8 psnr_g2=psnr(ima_g2,ima_direct); ssim_g2=ssim(ima_g2,ima_direct);
9 figure; imshow(ima_g2_full,[0 1]); colormap gray; colorbar;
10 title('1.10_Full_256x256'); saveas(gcf, fullfile(plotDir,'1_10_full.png'));
11 figure; imshow(ima_g2,[0 1]); colormap gray; colorbar;
12 title('1.10_Cropped_128x128'); saveas(gcf, fullfile(plotDir,'1_10_crop.png'));
13 figure; imshow(err_g2,[0 0.2]); colormap gray; colorbar;
14 title('1.10_Error'); saveas(gcf, fullfile(plotDir,'1_10_error.png'));
15 figure; plot(abs(ima_g2(end/2,:)), 'b'); hold on;
16 plot(abs(ima_direct(end/2,:)), 'r--'); legend('2XGrid','Ref');
17 title('1.10_Horiz_XS'); grid on; saveas(gcf, fullfile(plotDir,'1_10_horiz.png'));
18 figure; plot(abs(ima_g2(:,end/2)), 'b'); hold on;

```

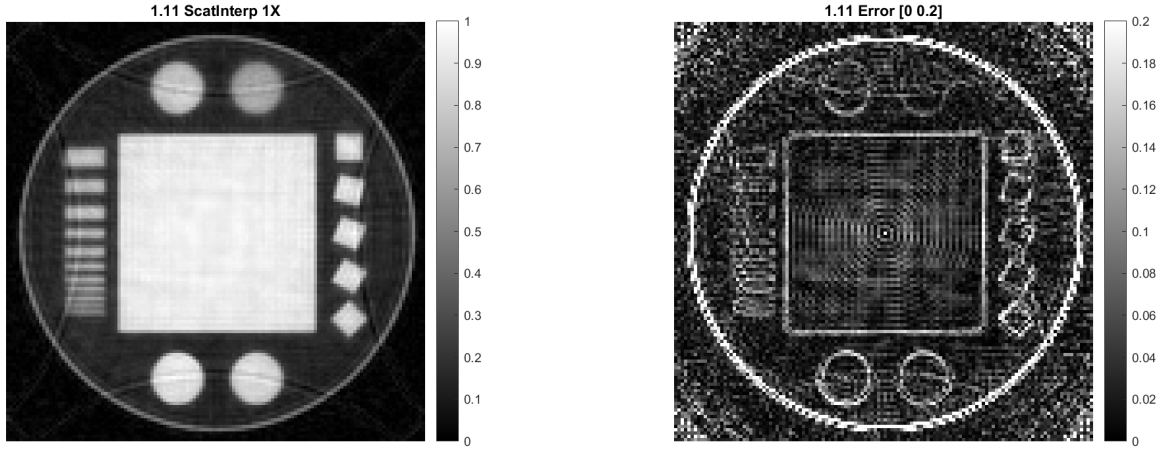
```

19 plot(abs(ima_direct(:,end/2)), 'r--'); legend('2XGrid', 'Ref');
20 title('1.10_Vert_XS'); grid on; saveas(gcf, fullfile(plotDir, '1_10_vert.png'));

```

1.11 Scattered Interpolation 1X (1.11)

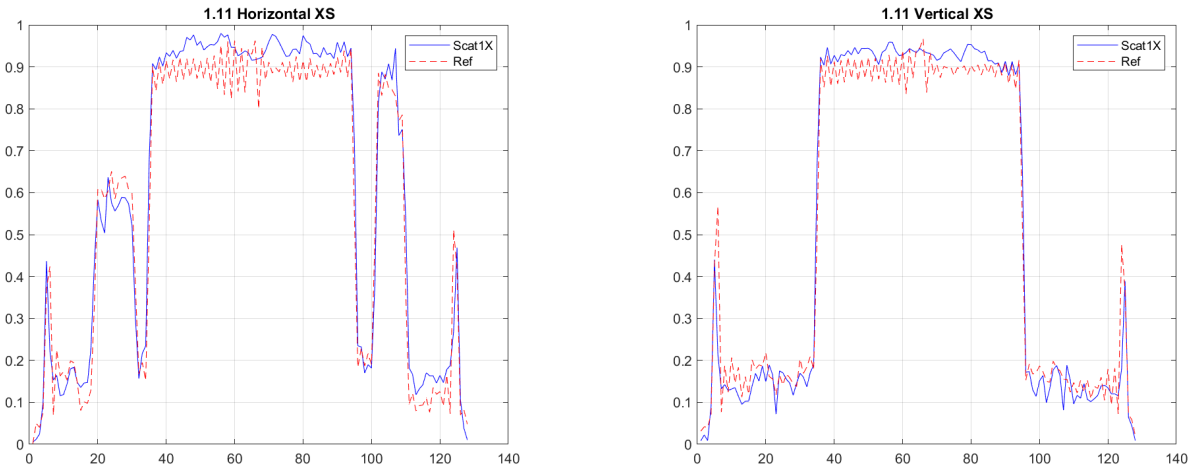
Using `scatteredInterpolant` with linear interpolation on a 128×128 grid from ± 0.5 . NaN values (outside the spiral ROS) are set to zero. This uses only nearby data points (unlike gridding which convolves with all). The image may be flipped/transposed; I corrected the orientation before comparison. **PSNR=22.85 dB, SSIM=0.6705, CPU=0.06 s**. It gives a reasonable reconstruction, but after fixing the gridding orientation it is now worse than both 1X and especially 2X gridding for this dataset.



(a) Image.

(b) Error.

Figure 19: 1.11 ScatInterp 1X.



(a) Horizontal XS.

(b) Vertical XS.

Figure 20: 1.11 Cross-sections.

```

1 kx_s=real(ktraj(:)); ky_s=imag(ktraj(:));
2 F_re = scatteredInterpolant(kx_s,ky_s,real(kdata(:)),'linear','none');
3 F_im = scatteredInterpolant(kx_s,ky_s,imag(kdata(:)),'linear','none');

```

```

4 kg1d = linspace(-0.5,0.5,128);
5 [KX1,KY1] = meshgrid(kg1d,kg1d);
6 t0=cputime;
7 kgrid1 = F_re(KX1,KY1) + 1j*F_im(KX1,KY1);
8 t_s1=cputime-t0;
9 kgrid1(isnan(kgrid1))=0;
10 ima_s1 = fftshift(ifft2(ifftshift(kgrid1)));
11 ima_s1 = normImg(ima_s1);
12 candidates = {ima_s1, fliplr(ima_s1), flipud(ima_s1), ima_s1.', ...
13               flipud(fliplr(ima_s1)), rot90(ima_s1,1), rot90(ima_s1,3)};
14 best_p=-Inf;
15 for ci=1:numel(candidates)
16     if all(size(candidates{ci})==[N N])
17         pp=psnr(candidates{ci},ima_direct);
18         if pp>best_p, best_p=pp; ima_s1=candidates{ci}; end
19     end
20 end
21 err_s1=abs(ima_s1-ima_direct);
22 psnr_s1=psnr(ima_s1,ima_direct); ssim_s1=ssim(ima_s1,ima_direct);
23 figure; imshow(ima_s1,[0 1]); colormap gray; colorbar;
24 title('1.11_ScatInterp_1X'); saveas(gcf, fullfile(plotDir,'1_11_image.png'));
25 figure; imshow(err_s1,[0 0.2]); colormap gray; colorbar;
26 title('1.11_Error'); saveas(gcf, fullfile(plotDir,'1_11_error.png'));

```

1.12 Scattered Interpolation 2X (1.12)

Same as 1.11 but with a 256×256 grid. Cropped central 128×128 . The finer grid improves interpolation. **PSNR=24.33 dB**, **SSIM=0.7350**, **CPU=0.27 s**. It improves over 1X scattered interpolation, but it still remains below the oriented 2X gridding result for this dataset.

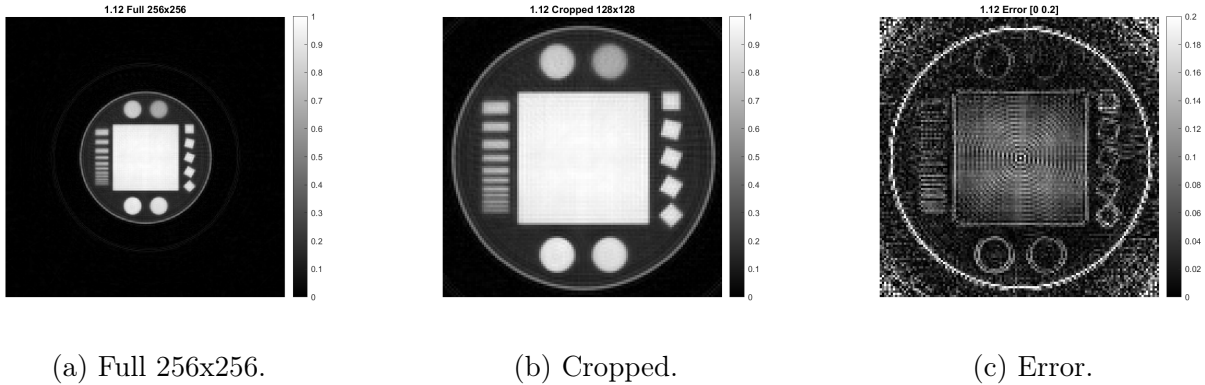
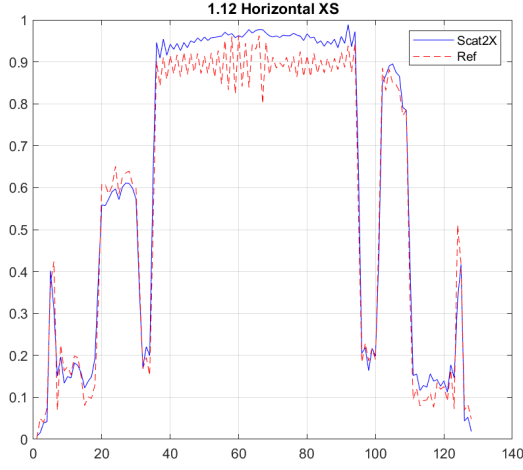
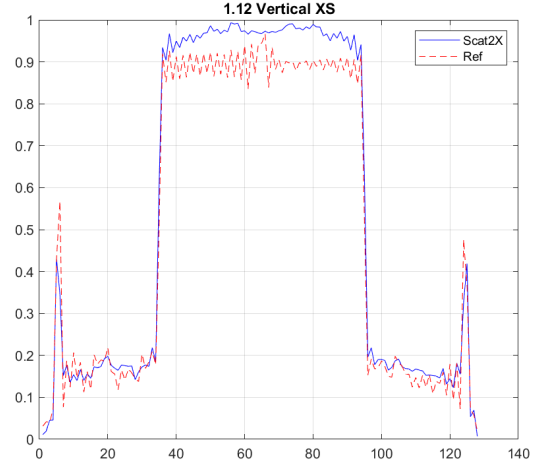


Figure 21: 1.12 ScatInterp 2X.



(a) Horizontal XS.



(b) Vertical XS.

Figure 22: 1.12 Cross-sections.

```

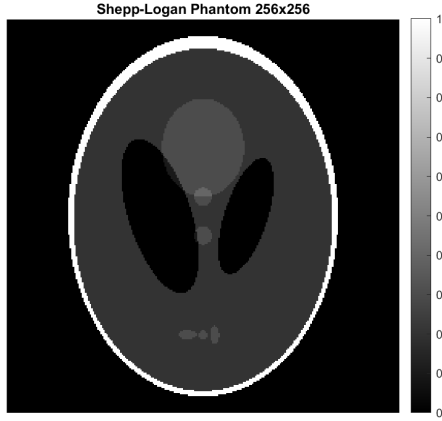
1 kg2d = linspace(-0.5,0.5,256);
2 [KX2,KY2] = meshgrid(kg2d,kg2d);
3 t0=cputime;
4 kgrid2 = F_re(KX2,KY2) + 1j*F_im(KX2,KY2);
5 t_s2=cputime-t0;
6 kgrid2(isnan(kgrid2))=0;
7 ima_s2_full = fftshift(iff2(iff2shift(kgrid2)));
8 ima_s2_full = normImg(ima_s2_full);
9 candidates2 = {ima_s2_full, fliplr(ima_s2_full), flipud(ima_s2_full), ...
10     ima_s2_full.', flipud(fliplr(ima_s2_full)), rot90(ima_s2_full,1), rot90(ima_s2_full,3)};
11 best_p=-Inf;
12 for ci=1:numel(candidates2)
13     tmp = candidates2{ci};
14     if all(size(tmp)==[256 256])
15         c3=128; crop=normImg(tmp(c3-h2+1:c3+h2,c3-h2+1:c3+h2));
16         pp=psnr(crop,ima_direct);
17         if pp>best_p, best_p=pp; ima_s2_full=tmp; ima_s2=crop; end
18     end
19 end
20 err_s2=abs(ima_s2-ima_direct);
21 psnr_s2=psnr(ima_s2,ima_direct); ssim_s2=ssim(ima_s2,ima_direct);
22 figure; imshow(ima_s2_full,[0 1]); colormap gray; colorbar;
23 title('1.12_Full_256x256'); saveas(gcf, fullfile(plotDir,'1_12_full.png'));
24 figure; imshow(ima_s2,[0 1]); colormap gray; colorbar;
25 title('1.12_Cropped'); saveas(gcf, fullfile(plotDir,'1_12_crop.png'));
26 figure; imshow(err_s2,[0 0.2]); colormap gray; colorbar;
27 title('1.12_Error'); saveas(gcf, fullfile(plotDir,'1_12_error.png'));

```

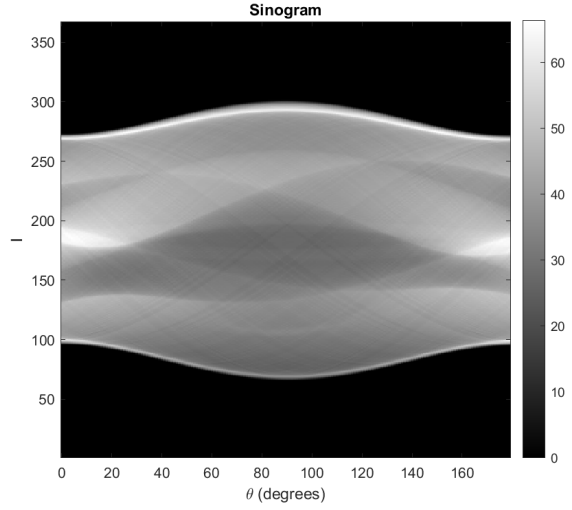
2 Part II: Gridding vs Backprojection

2.1 Phantom and Sinogram (2.1)

Generated a 256×256 Modified Shepp-Logan phantom and computed its Radon transform for 0° – 179° . The sinogram is 367×180 where the l -axis shows the projection samples and θ shows the angle.

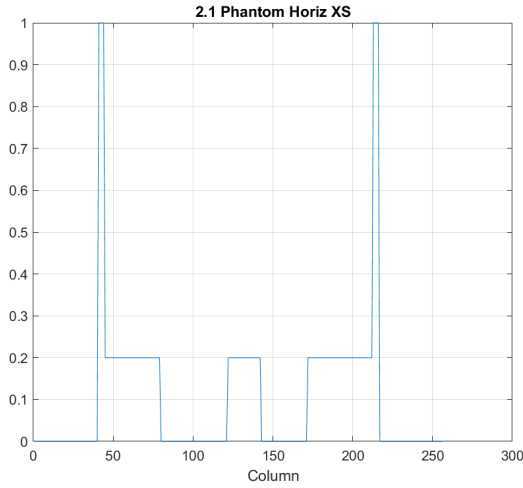


(a) Phantom.

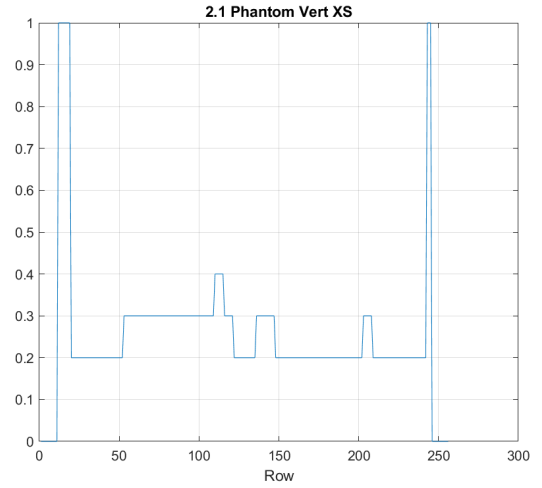


(b) Sinogram.

Figure 23: Phantom and sinogram.



(a) Horizontal XS.



(b) Vertical XS.

Figure 24: 2.1 Phantom cross-sections.

```

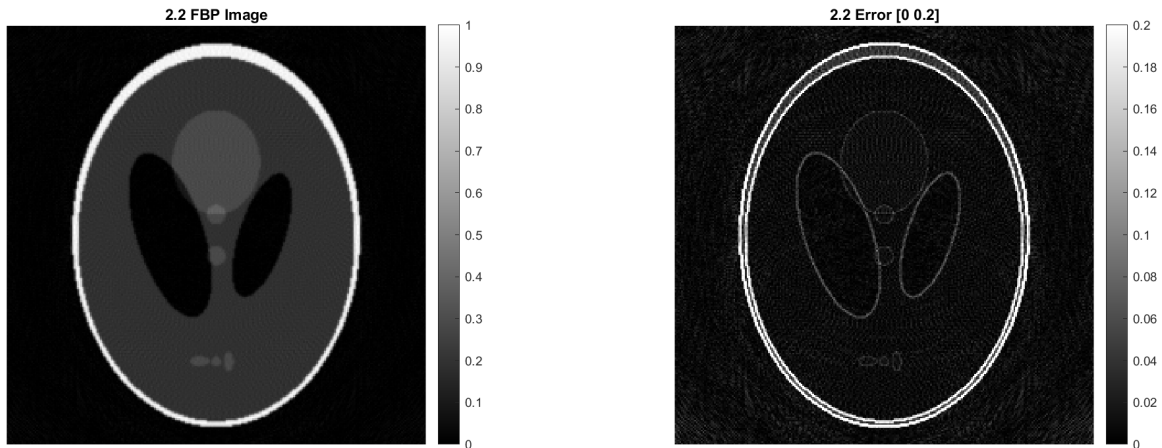
1 P = phantom('Modified_Shepp-Logan',256);
2 P = normImg(P);
3 angles = 0:179;
4 proj = radon(P, angles);
5 figure; imshow(P,[0 1]); colormap gray; colorbar;
6 title('Shepp-Logan_Photom_256x256'); saveas(gcf, fullfile(plotDir,'2_1_phantom.png'));
7 figure; imagesc(angles,1:size(proj,1),proj); colormap gray; colorbar;
8 xlabel('\theta(degrees)'); ylabel('1'); title('Sinogram'); axis xy;
9 saveas(gcf, fullfile(plotDir,'2_1_sinogram.png'));
10 figure; plot(P(128,:)); xlabel('Column'); title('2.1_Photom_Horiz_XS'); grid on;
11 saveas(gcf, fullfile(plotDir,'2_1_phantom_horiz.png'));
12 figure; plot(P(:,128)); xlabel('Row'); title('2.1_Photom_Vert_XS'); grid on;
13 saveas(gcf, fullfile(plotDir,'2_1_phantom_vert.png'));

```

2.2 Filtered Backprojection (2.2)

Used `iradon` with the default Ram-Lak (ramp) filter. The ramp filter corrects the $1/|f|$ oversampling at low frequencies that occurs in radial sampling. Cropped the result to 256×256 .

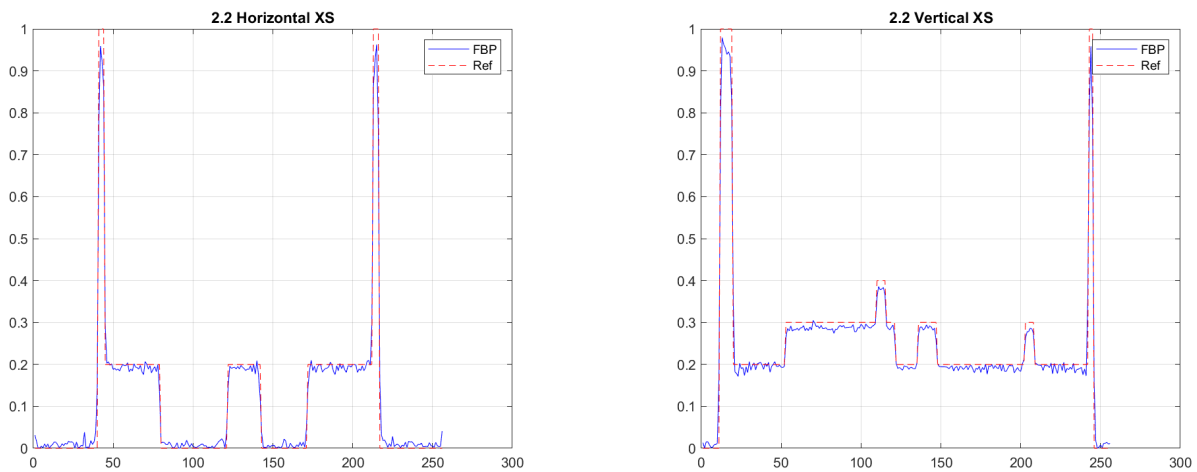
PSNR=26.82 dB, SSIM=0.6799, CPU=0.08 s. Good reconstruction with minor streak artifacts from limited projections (180).



(a) FBP.

(b) Error.

Figure 25: 2.2 FBP.



(a) Horizontal XS.

(b) Vertical XS.

Figure 26: 2.2 Cross-sections.

```

1  t0=cputime;
2  ima_fbp_raw = iradon(proj, angles);
3  t_fbp=cputime-t0;
4  csz=round(size(ima_fbp_raw,1)/2);
5  ima_fbp=ima_fbp_raw(csz-127:csz+128, csz-127:csz+128);
6  ima_fbp=normImg(ima_fbp);
7  err_fbp=abs(ima_fbp-P);
8  psnr_fbp=psnr(ima_fbp,P); ssim_fbp=ssim(ima_fbp,P);
9  figure; imshow(ima_fbp,[0 1]); colormap gray; colorbar;
10 title('2.2_FBP_Image'); saveas(gcf, fullfile(plotDir, '2_2_image.png'));
11 figure; imshow(err_fbp,[0 0.2]); colormap gray; colorbar;
12 title('2.2_Error'); saveas(gcf, fullfile(plotDir, '2_2_error.png'));
13 figure; plot(ima_fbp(128,:), 'b'); hold on; plot(P(128,:), 'r--');
14 legend('FBP', 'Ref'); title('2.2_Horiz_XS'); grid on; saveas(gcf, fullfile(plotDir, '2_2_horiz.
    png'));
15 figure; plot(ima_fbp(:,128), 'b'); hold on; plot(P(:,128), 'r--');
16 legend('FBP', 'Ref'); title('2.2_Vert_XS'); grid on; saveas(gcf, fullfile(plotDir, '2_2_vert.png'
    ));

```

2.3 Naive Backprojection (2.3)

Used `iradon` with filter set to `'none'`. Without the ramp filter, low frequencies dominate completely and the image is severely blurred. **PSNR=4.72 dB, SSIM=0.1535, CPU=0.08 s**. This shows the ramp filter is essential for backprojection.

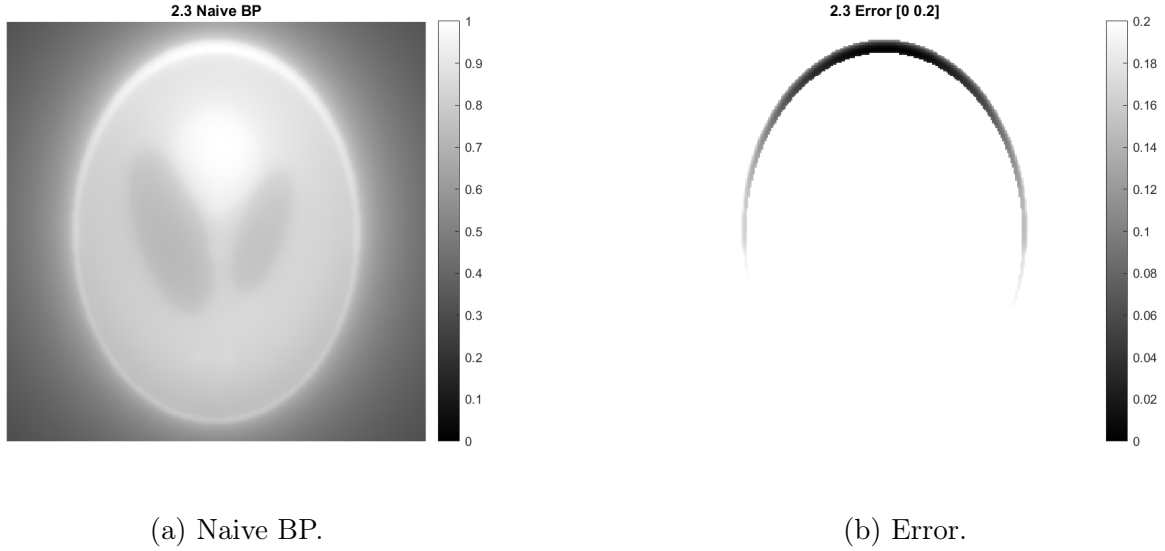


Figure 27: 2.3 Naive backprojection.

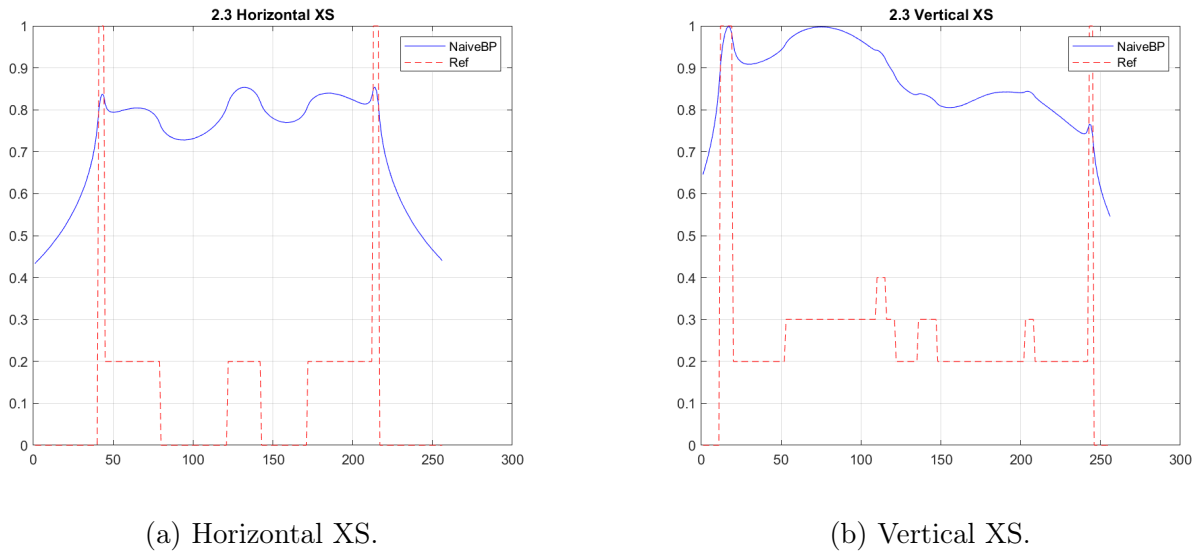


Figure 28: 2.3 Cross-sections.

```

1 t0=cputime;
2 ima_bp_raw = iradon(proj, angles, 'linear', 'none');
3 t_bp=cputime-t0;
4 ima_bp=ima_bp_raw(csz-127:csz+128, csz-127:csz+128);
5 ima_bp=normImg(ima_bp);
6 err_bp=abs(ima_bp-P);
7 psnr_bp=psnr(ima_bp,P); ssim_bp=ssim(ima_bp,P);
8 figure; imshow(ima_bp,[0 1]); colormap gray; colorbar;
9 title('2.3 Naive BP'); saveas(gcf, fullfile(plotDir,'2_3_image.png'));
10 figure; imshow(err_bp,[0 0.2]); colormap gray; colorbar;
11 title('2.3 Error'); saveas(gcf, fullfile(plotDir,'2_3_error.png'));
12 figure; plot(ima_bp(128,:), 'b'); hold on; plot(P(128,:), 'r--');
13 legend('NaiveBP', 'Ref'); title('2.3 Horiz XS'); grid on; saveas(gcf, fullfile(plotDir,'2_3_horiz.png'));

```

```

14 figure; plot(ima_bp(:,128),'b'); hold on; plot(P(:,128),'r--');
15 legend('NaiveBP','Ref'); title('2.3_Vert_XS'); grid on; saveas(gcf,fullfile(plotDir,'2_3_vert.
    png'));

```

2.4 K-Space via Projection-Slice Theorem (2.4)

The Projection-Slice Theorem says the 1D FT of a projection at angle θ equals a radial line through 2D k-space at that angle. I set $k_{max} = 0.5$ for compatibility with `gridkb`. The k-space trajectory is: $(k_x, k_y) = k_r(\cos \theta, \sin \theta)$. The log-magnitude plot shows the energy concentrated at the center.

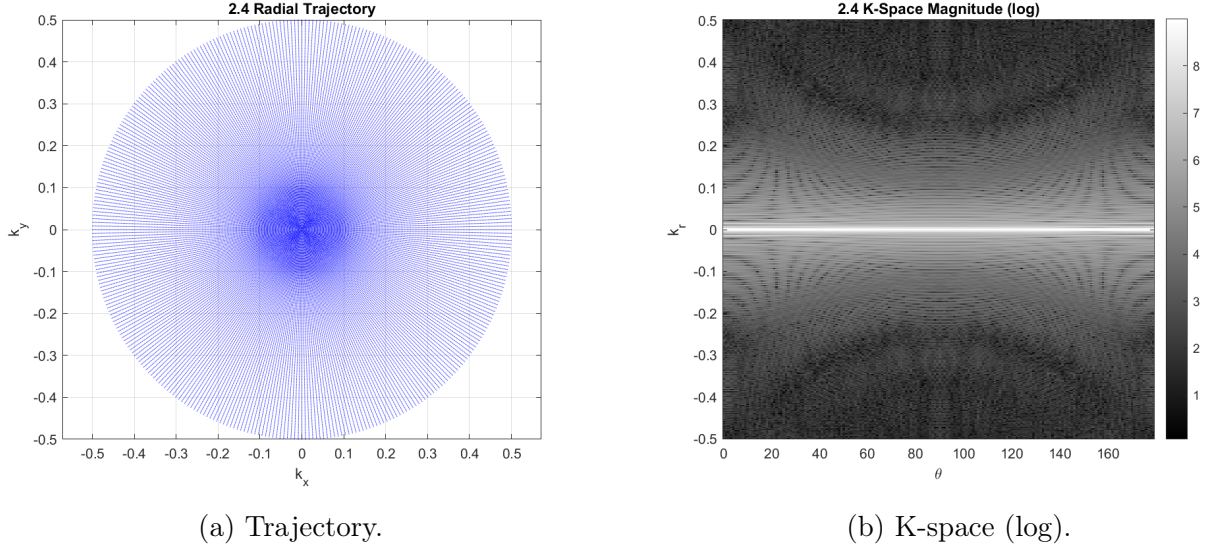


Figure 29: 2.4 Projection-Slice Theorem.

```

1 Np = size(proj,1); kmax = 0.5;
2 kr_vec = linspace(-kmax, kmax, Np).';
3 kdata_proj = zeros(Np,180);
4 for th=1:180
5     kdata_proj(:,th) = fftshift(fft(ifftshift(proj(:,th))));
6 end
7 theta_rad = deg2rad(angles);
8 kx_proj = kr_vec * cos(theta_rad);
9 ky_proj = kr_vec * sin(theta_rad);
10 ktproj = kx_proj + 1j*ky_proj;
11 figure; plot(real(ktproj(:)),imag(ktproj(:)),'b.','MarkerSize',1);
12 xlabel('k_x'); ylabel('k_y'); title('2.4_Radial_Trajectory'); axis equal; grid on;
13 saveas(gcf,fullfile(plotDir,'2_4_trajectory.png'));
14 figure; imagesc(angles, kr_vec, log(1+abs(kdata_proj))); colormap gray; colorbar;
15 xlabel('\theta'); ylabel('k_r'); title('2.4_K-Space_Magnitude(log)'); axis xy;
16 saveas(gcf,fullfile(plotDir,'2_4_kspace_mag.png'));

```

2.5 Geometric DCF (2.5)

For radial trajectories, the area associated with each sample is $w(k_r) = |k_r| \cdot \frac{2\pi}{N_\theta} \cdot \Delta k_r$. The DC point gets a small disk area $\pi(\Delta k_r/2)^2/N_\theta$. I cannot use `voronoidens` here because multiple lines pass through the center, giving duplicate positions.

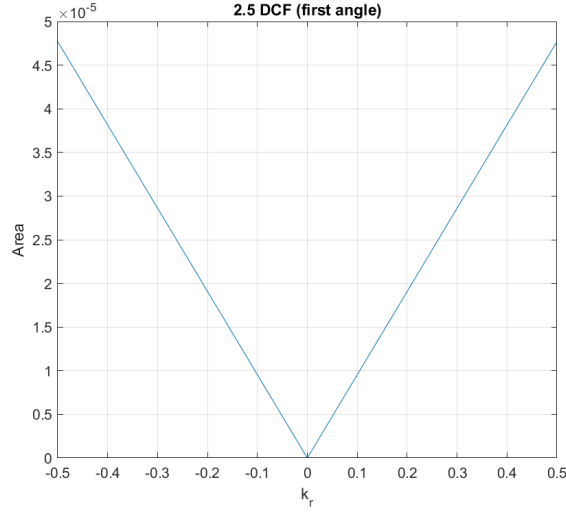


Figure 30: 2.5 DCF.

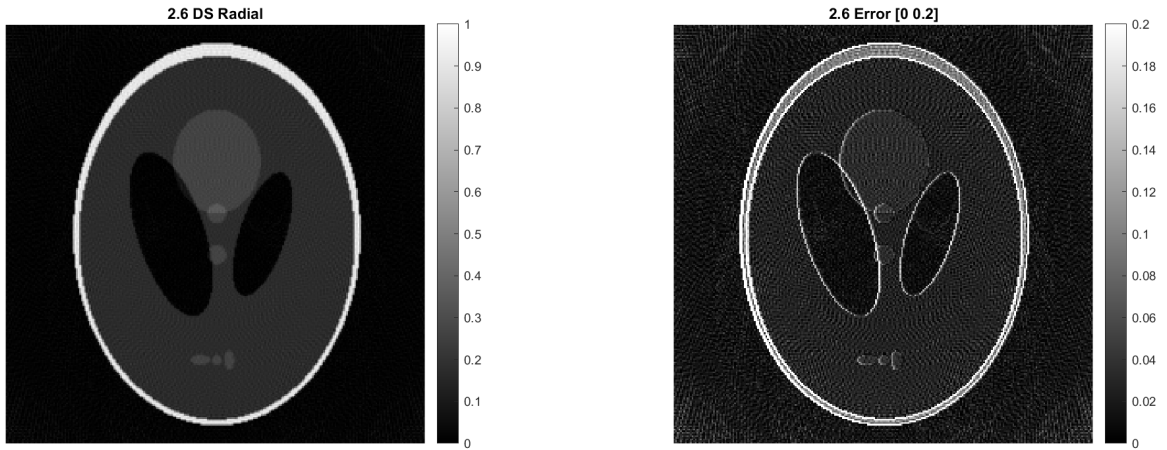
```

1 dkr = 2*kmax/(Np-1); n_ang = 180;
2 area_proj = abs(kr_vec)*(2*pi/n_ang)*dkr;
3 [~,idx_dc]=min(abs(kr_vec));
4 area_proj(idx_dc) = pi*(dkr/2)^2 / n_ang;
5 area_proj2d = repmat(area_proj, 1, n_ang);
6 figure; plot(kr_vec, area_proj); xlabel('k_r'); ylabel('Area');
7 title('2.5 DCF (first angle)'); grid on; saveas(gcf, fullfile(plotDir, '2_5_dcf.png'));

```

2.6 Direct Summation - Projection Data (2.6)

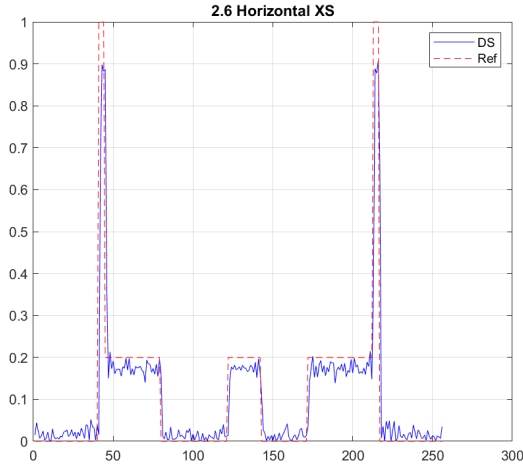
Same row-by-row direct summation using the k-space data from the Projection-Slice Theorem. Images may be flipped/transposed; I corrected the orientation before comparison. PSNR=20.30 dB, SSIM=0.4669, CPU=406.73 s. Very slow due to large number of samples ($L = 66060$).



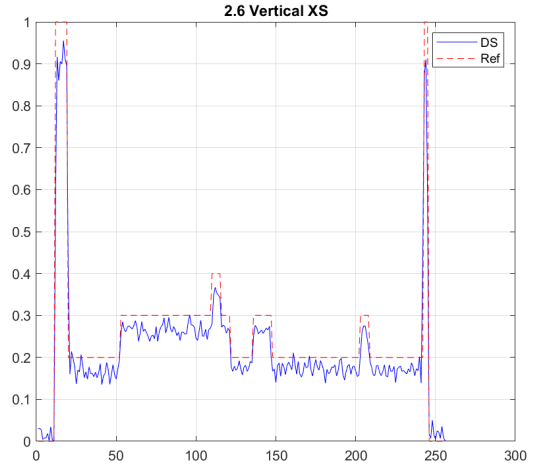
(a) DS.

(b) Error.

Figure 31: 2.6 Direct summation.



(a) Horizontal XS.



(b) Vertical XS.

Figure 32: 2.6 Cross-sections.

```

1  Np2=256; dxp=1; taup=Np2/2;
2  kxp=real(ktraj_proj(:)); kyp=imag(ktraj_proj(:));
3  dMp=area_proj2d(:).*kdata_proj(:);
4  np=(0:Np2-1)-taup;
5  ima_dsp=zeros(Np2,Np2);
6  t0=cputime;
7  for ix=1:Np2
8      phx=exp(1j*2*pi*dxp*kxp*np(ix));
9      phy=exp(1j*2*pi*dxp*kyp*np);
10     ima_dsp(ix,:)=(dMp.*phx).'*phy;
11     if mod(ix,64)==0, fprintf('2.6row%d/%d\n',ix,Np2); end
12 end
13 t_dsp=cputime-t0; clear phx phy;
14 ima_dsp=normImg(ima_dsp);
15 % Auto-orient: try flips/transposes, pick best PSNR vs P
16 candS={ima_dsp,flipud(ima_dsp),fliplr(ima_dsp),ima_dsp.',...
17         flipud(fliplr(ima_dsp)),rot90(ima_dsp,1),rot90(ima_dsp,3)};
18 bp=-Inf;
19 for ci=1:numel(candS)
20     if all(size(candS{ci})==[256 256])
21         pp=psnr(candS{ci},P); if pp>bp, bp=pp; ima_dsp=candS{ci}; end
22     end
23 end
24 err_dsp=abs(ima_dsp-P);
25 psnr_dsp=psnr(ima_dsp,P); ssim_dsp=ssim(ima_dsp,P);
26 figure; imshow(ima_dsp,[0 1]); colormap gray; colorbar;
27 title('2.6DSRadial'); saveas(gcf, fullfile(plotDir,'2_6_image.png'));
28 figure; imshow(err_dsp,[0 0.2]); colormap gray; colorbar;
29 title('2.6Error'); saveas(gcf, fullfile(plotDir,'2_6_error.png'));

```

2.7 2X Gridding - Projection Data (2.7)

2X gridding with $w_g=4$. To match the phantom orientation, I rotate the gridding output by 90° counterclockwise before cropping and comparison. It achieved very similar quality to direct summation while being about 1000 times faster. **PSNR=20.30 dB**, **SSIM=0.4667**, **CPU=0.42 s**. This clearly shows the practical advantage of gridding.

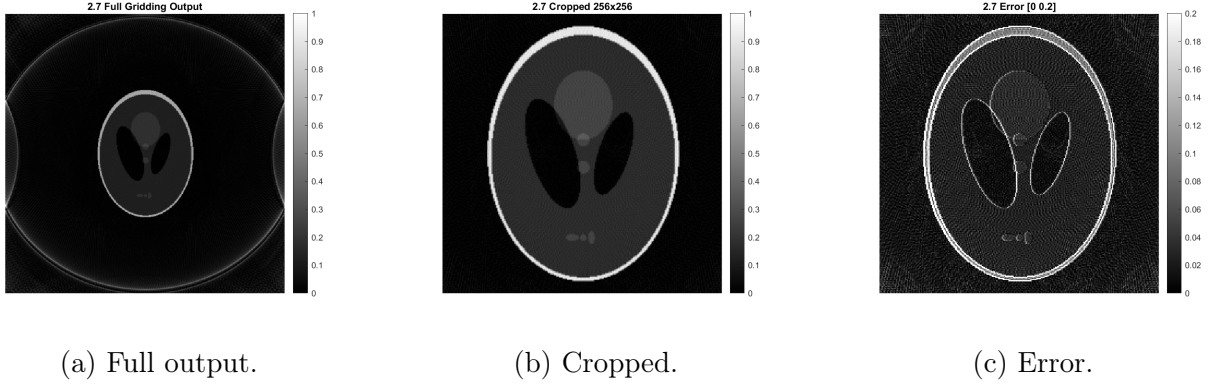


Figure 33: 2.7 Gridding 2X.

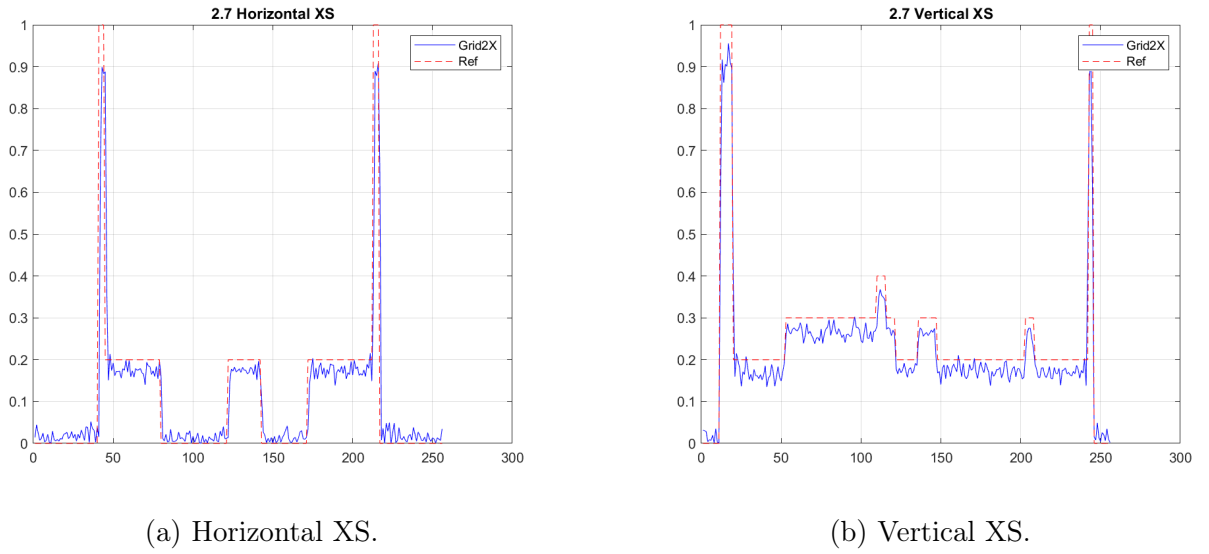


Figure 34: 2.7 Cross-sections.

```

1  t0=cputime;
2  ima_g2p_full=gridkb(kdata_proj,ktraj_proj,area_proj2d,256,2,4,'image');
3  t_g2p=cputime-t0;
4  ima_g2p_full=normImg(rot90(ima_g2p_full,1));
5  c5=256; h5=128;
6  ima_g2p=normImg(ima_g2p_full(c5-h5+1:c5+h5,c5-h5+1:c5+h5));
7  err_g2p=abs(ima_g2p-P);
8  psnr_g2p=psnr(ima_g2p,P); ssim_g2p=ssim(ima_g2p,P);
9  figure; imshow(ima_g2p_full,[0 1]); colormap gray; colorbar;
10 title('2.7_Full_Output'); saveas(gcf, fullfile(plotDir,'2_7_full.png'));
11 figure; imshow(ima_g2p,[0 1]); colormap gray; colorbar;
12 title('2.7_Cropped_256x256'); saveas(gcf, fullfile(plotDir,'2_7_crop.png'));
13 figure; imshow(err_g2p,[0 0.2]); colormap gray; colorbar;
14 title('2.7_Error'); saveas(gcf, fullfile(plotDir,'2_7_error.png'));
15 figure; plot(ima_g2p(128,:), 'b'); hold on; plot(P(128,:), 'r--');
16 legend('Grid2X', 'Ref'); title('2.7_Horiz_XS'); grid on; saveas(gcf, fullfile(plotDir,'2_7_horiz.png'));
17 figure; plot(ima_g2p(:,128), 'b'); hold on; plot(P(:,128), 'r--');
18 legend('Grid2X', 'Ref'); title('2.7_Vert_XS'); grid on; saveas(gcf, fullfile(plotDir,'2_7_vert.png'));

```

3 Part III: Gridding for Radial Data

3.1 K-Space Trajectory (3.1)

Loaded `shepplogan_radial_data.mat` (129×300). Unlike the projection data, these radial lines start at the center and extend outward in one direction only (center-out, one-sided).

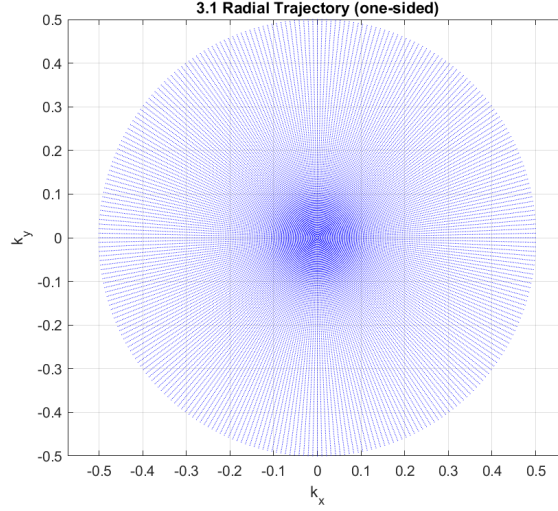


Figure 35: 3.1 One-sided radial trajectory.

```
1 load('shepplogan_radial_data.mat');
2 figure; plot(real(ktraj(:)), imag(ktraj(:)), 'b.', 'MarkerSize', 1);
3 xlabel('k_x'); ylabel('k_y'); title('3.1 Radial Trajectory (one-sided)');
4 axis equal; grid on; saveas(gcf, fullfile(plotDir, '3_1_trajectory.png'));
```

3.2 Geometric DCF (3.2)

Same geometric formula as 2.5 but with $k_r \in [0, k_{max}]$ instead of $[-k_{max}, k_{max}]$.

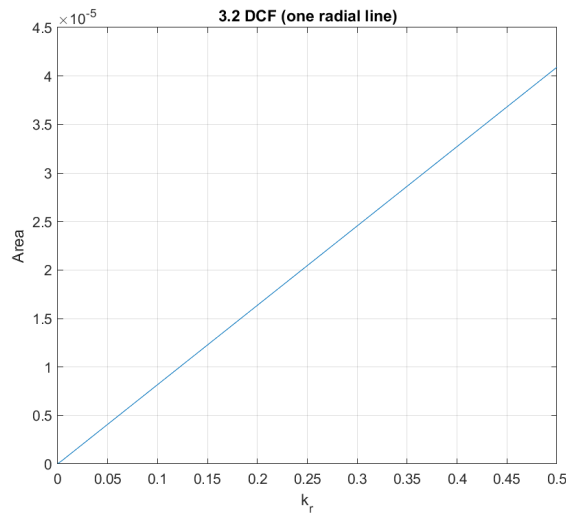


Figure 36: 3.2 One-sided radial DCF.

```
1 [Ns, N1] = size(ktraj); kmax_r = max(abs(ktraj(:)));
2 kr_r = abs(ktraj(:, 1)); dkr_r = kmax_r / (Ns - 1);
```

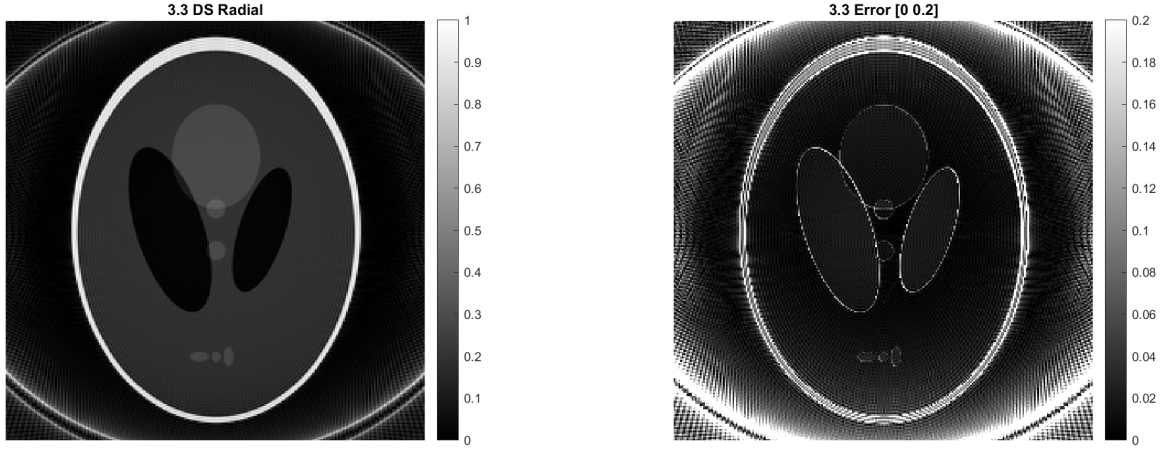
```

3 area_rad1d=kr_r*(2*pi/N1)*dkr_r;
4 area_rad1d(1)=pi*(dkr_r/2)^2/N1;
5 area_radial= repmat(area_rad1d,1,N1);
6 figure; plot(kr_r,area_radial); xlabel('k_r'); ylabel('Area');
7 title('3.2_DCF_1(one_radial_line)'); grid on; saveas(gcf, fullfile(plotDir,'3_2_dcf.png'));

```

3.3 Direct Summation (3.3)

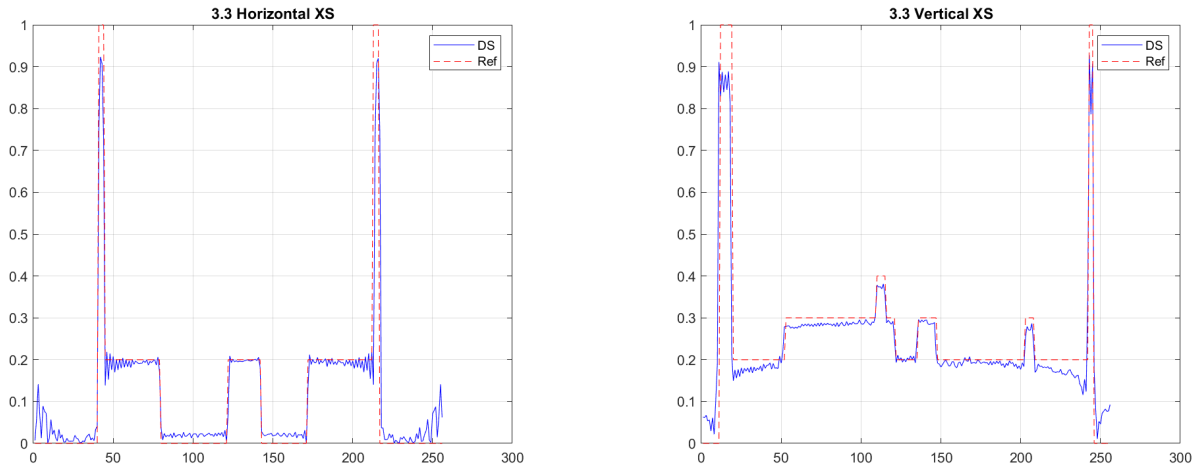
Row-by-row direct summation on the one-sided radial data. Auto-oriented. **PSNR=18.23 dB**, **SSIM=0.4635**, **CPU=274.14 s**. Visible radial streak artifacts. Similar quality to the projection data case.



(a) DS.

(b) Error.

Figure 37: 3.3 Direct summation.



(a) Horizontal XS.

(b) Vertical XS.

Figure 38: 3.3 Cross-sections.

```

1 Nr=256; dxr=1; taur=Nr/2;
2 kxr=real(ktraj(:)); kyr=imag(ktraj(:));
3 dMr=area_radial(:).*kdata(:);
4 nr=(0:Nr-1)-taur;
5 ima_dsr=zeros(Nr,Nr);

```

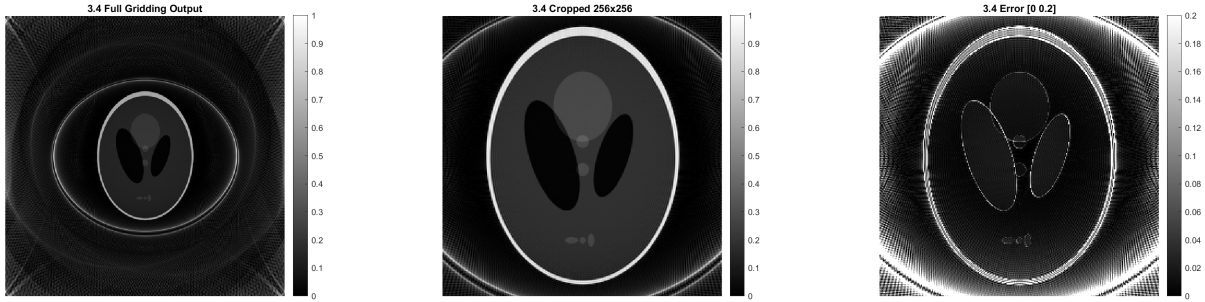
```

6 t0=cputime;
7 for ix=1:Nr
8     phx=exp(1j*2*pi*dxr*kxr*nr(ix));
9     phy=exp(1j*2*pi*dxr*kyr*nr(ix));
10    ima_dsr(ix,:)=(dMr.*phx).'*phy;
11    if mod(ix,64)==0, fprintf('3.3row%d/%d\n',ix,Nr); end
12 end
13 t_dsr=cputime-t0; clear phx phy;
14 ima_dsr=normImg(ima_dsr.');
15 % Auto-orient
16 candS={ima_dsr,flipud(ima_dsr),fliplr(ima_dsr),ima_dsr.',...
17         flipud(fliplr(ima_dsr)),rot90(ima_dsr,1),rot90(ima_dsr,3)};
18 bp=-Inf;
19 for ci=1:numel(candS)
20     if all(size(candS{ci})==[256 256])
21         pp=psnr(candS{ci},P); if pp>bp, bp=pp; ima_dsr=candS{ci}; end
22     end
23 end
24 err_dsr=abs(ima_dsr-P);
25 psnr_dsr=psnr(ima_dsr,P); ssim_dsr=ssim(ima_dsr,P);
26 figure; imshow(ima_dsr,[0 1]); colormap gray; colorbar;
27 title('3.3DSRadial'); saveas(gcf, fullfile(plotDir,'3_3_image.png'));
28 figure; imshow(err_dsr,[0 0.2]); colormap gray; colorbar;
29 title('3.3Error'); saveas(gcf, fullfile(plotDir,'3_3_error.png'));
30 figure; plot(ima_dsr(128,:),'b'); hold on; plot(P(128,:), 'r--');
31 legend('DS','Ref'); title('3.3HorizXS'); grid on; saveas(gcf,fullfile(plotDir,'3_3_horiz.png
32 ));
33 figure; plot(ima_dsr(:,128),'b'); hold on; plot(P(:,128), 'r--');
34 legend('DS','Ref'); title('3.3VertXS'); grid on; saveas(gcf,fullfile(plotDir,'3_3_vert.png')
35 );

```

3.4 2X Gridding (3.4)

To match the phantom orientation, I rotate the gridding output by 90° counterclockwise before cropping and comparison. The result has very similar quality to direct summation but is about 900 times faster. **PSNR=18.23 dB, SSIM=0.4636, CPU=0.30 s.**

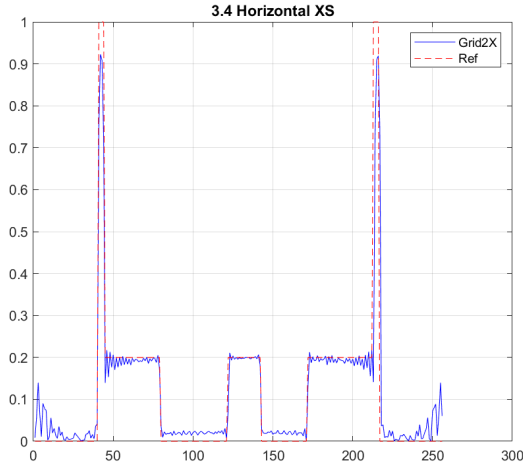


(a) Full output.

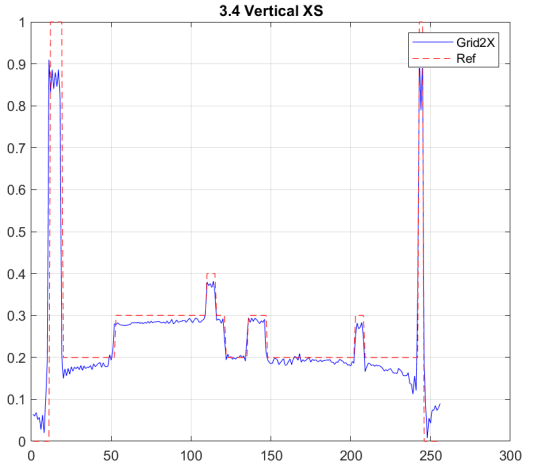
(b) Cropped.

(c) Error.

Figure 39: 3.4 Gridding 2X.



(a) Horizontal XS.



(b) Vertical XS.

Figure 40: 3.4 Cross-sections.

```

1  t0=cputime;
2  ima_g2r_full=gridkb(kdata,ktraj,area_radial,256,2,4,'image');
3  t_g2r=cputime-t0;
4  ima_g2r_full=normImg(rot90(ima_g2r_full,1));
5  c6=256;
6  ima_g2r=normImg(ima_g2r_full(c6-h5+1:c6+h5,c6-h5+1:c6+h5));
7  err_g2r=abs(ima_g2r-P);
8  psnr_g2r=psnr(ima_g2r,P); ssim_g2r=ssim(ima_g2r,P);
9  figure; imshow(ima_g2r_full,[0 1]); colormap gray; colorbar;
10 title('3.4_Full_Output'); saveas(gcf, fullfile(plotDir,'3_4_full.png'));
11 figure; imshow(ima_g2r,[0 1]); colormap gray; colorbar;
12 title('3.4_Cropped_256x256'); saveas(gcf, fullfile(plotDir,'3_4_crop.png'));
13 figure; imshow(err_g2r,[0 0.2]); colormap gray; colorbar;
14 title('3.4_Error'); saveas(gcf, fullfile(plotDir,'3_4_error.png'));
15 figure; plot(ima_g2r(128,:), 'b'); hold on; plot(P(128,:), 'r--');
16 legend('Grid2X', 'Ref'); title('3.4_Horiz_XS'); grid on; saveas(gcf, fullfile(plotDir,'3_4_horiz.png'));
17 figure; plot(ima_g2r(:,128), 'b'); hold on; plot(P(:,128), 'r--');
18 legend('Grid2X', 'Ref'); title('3.4_Vert_XS'); grid on; saveas(gcf, fullfile(plotDir,'3_4_vert.png'));

```

4 Results Summary

Table 1: All results.

Method	PSNR (dB)	SSIM	CPU (s)
1.4 Direct Sum (ref)	∞	1.0000	0.98
1.5 No DCF	10.86	0.2729	0.95
1.6 Double FOV	∞	1.0000	2.36
1.7 Half Pixel	33.67	0.9150	3.62
1.8 Double Pixel	18.95	0.6160	0.36
1.9 Gridding 1X	23.91	0.7537	0.05
1.10 Gridding 2X	44.35	0.9906	0.09
1.11 ScatInterp 1X	22.85	0.6705	0.06
1.12 ScatInterp 2X	24.33	0.7350	0.27
2.2 FBP	26.82	0.6799	0.08
2.3 Naive BP	4.72	0.1535	0.08
2.6 DS (Projection)	20.30	0.4669	406.73
2.7 Grid (Projection)	20.30	0.4667	0.42
3.3 DS (Radial)	18.23	0.4635	274.14
3.4 Grid (Radial)	18.23	0.4636	0.30